

گزارش تمرین کامپیوتری شماره ۱

شبیه سازی پروتکل های ارتباطی

درس سیستم های نهفته بی درنگ

نیم سال دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

موضوع گزارش: پیاده سازی و تکمیل ارتباط های C2I/I2C، USART و I2C MUX در شبیه ساز

زبان پیاده سازی: C++20

اعضای گروه

نام و نام خانوادگی	شماره دانشجویی
فرشته باقری	۸۱۰۱۰۰۰۸۹
محمد امانلو	۸۱۰۱۰۰۰۸۴
مهراد لیویان	۸۱۰۱۰۱۵۰۱
گلبو رشیدی	۸۱۰۱۰۰۱۴۸

فهرست مطالب

۶	۱	مقدمه
۶	۲	هدف کلی پروژه
۷	۳	معماری کلی شبیه ساز و کدهای آماده پروژه
۷	۱.۳	کلاس Object و نام گذاری اجزا
۷	۲.۳	کلاس های Pin و Link
۷	۳.۳	کلاس Board
۷	۴.۳	کلاس Processor
۸	۵.۳	کلاس های پایه پروتکل
۸	۶.۳	کلاس AbstractSketch
۸	۷.۳	ابزارهای کمکی آماده
۸	۴	تکمیل و استفاده از بورد ESP8266
۹	۵	تکمیل و استفاده از سنسور VL530X
۹	۶	پیاده سازی پروتکل I2C/C2I
۹	۱.۶	هدف از پیاده سازی
۹	۲.۶	ساختار کلی ارتباط
۱۰	۳.۶	ماشین حالت سمت Master
۱۰	۴.۶	ماشین حالت سمت Slave
۱۰	۷	تکمیل Sketch سنسور
۱۰	۱.۷	تابع setup سنسور
۱۰	۲.۷	تابع loop سنسور
۱۱	۳.۷	دلیل استفاده از checksum
۱۱	۸	تکمیل Sketch میکروکنترلر
۱۱	۱.۸	تابع setup میکروکنترلر
۱۱	۲.۸	تابع loop میکروکنترلر
۱۱	۳.۸	منطق تابع handle_usart
۱۲	۴.۸	همزمانی ارتباط I2C و USART
۱۲	۹	مدیریت داده چندبایتی در ارتباط با سنسور
۱۲	۱۰	پیاده سازی پروتکل USART
۱۲	۱.۱۰	هدف از پیاده سازی
۱۲	۲.۱۰	مقداردهی اولیه پروتکل
۱۳	۳.۱۰	ساختار فریم
۱۳	۴.۱۰	ماشین حالت دریافت
۱۳	۵.۱۰	ماشین حالت ارسال
۱۳	۶.۱۰	توابع read و write
۱۴	۱۱	پیاده سازی و استفاده از بورد Usb

۱۴	۱۲ تکمیل بخش ارتباطی HardDisk
۱۴	۱.۱۲ حافظه داخلی
۱۵	۲.۱۲ تابع setup
۱۵	۳.۱۲ تابع loop
۱۶	۱۳ اتصال اجزای سیستم در تابع main
۱۶	۱.۱۳ ساخت بوردها
۱۶	۲.۱۳ لینک های ارتباط I2C/C2I
۱۶	۳.۱۳ لینک های ارتباط USART
۱۶	۴.۱۳ اتصال پایه ها
۱۷	۵.۱۳ شروع اجرای Sketch ها
۱۷	۱۴ جریان کلی اجرای برنامه
۱۷	۱۵ مدیریت زمان و سیکل ها
۱۷	۱۶ طراحی خطایابی و پیام های خروجی
۱۸	۱.۱۶ پیام های راه اندازی
۱۸	۲.۱۶ پیام های ارتباط I2C/C2I
۱۸	۳.۱۶ پیام های ارتباط USART
۱۸	۱۷ سناریوهای تست و خروجی اجرا
۱۸	۱.۱۷ تست ارتباط I2C/C2I
۱۸	۲.۱۷ تست ارتباط USART
۱۹	۳.۱۷ نمونه خروجی اجرا
۲۰	۱۸ خروجی اختصاصی تست I2C/C2I و تحلیل آن
۲۰	۱.۱۸ نحوه پیاده سازی تست اختصاصی
۲۱	۲.۱۸ خروجی نهایی:
۲۴	۱۹ خروجی اختصاصی تست USART و تحلیل آن
۲۵	۱.۱۹ ساخت بوردها و اتصال پایه ها
۲۵	۲.۱۹ مقداردهی اولیه USART
۲۵	۱.۲.۱۹ پیاده سازی تست
۲۶	۳.۱۹ خروجی کامل تست USART
۲۷	۴.۱۹ تحلیل روند ارسال و دریافت آدرس ها
۲۸	۵.۱۹ نکته مربوط به ترتیب چاپ پیام ها
۲۸	۶.۱۹ جمع بندی تست مستقل USART
۲۸	۲۰ تحلیل خروجی اجرای برنامه در تست ترکیبی
۲۹	۱.۲۰ مرحله ساخت بوردها و اتصال لینک ها
۲۹	۲.۲۰ مرحله مقداردهی اولیه پروتکل ها
۳۰	۳.۲۰ تحلیل همزمانی ارتباط I2C/C2I و USART
۳۱	۴.۲۰ تحلیل داده های سنسور در خروجی I2C/C2I
۳۲	۵.۲۰ تحلیل خروجی های میانی و پیام های همزمان
۳۳	۶.۲۰ پایان اجرای USART و دریافت آخرین آدرس ها
۳۴	۷.۲۰ جمع بندی تحلیل خروجی

۳۵	بخش سوم: پیاده سازی و تست C2I/I2C Multiplexer
۳۵	۱.۲۱ هدف بخش سوم
۳۵	۲.۲۱ نکته مهم درباره پروتکل ها
۳۵	۳.۲۱ مدل سخت افزاری MUX
۳۶	۴.۲۱ کلاس I2CMux
۳۶	۵.۲۱ منطق Sketch مربوط به MUX
۳۷	۶.۲۱ منطق Sketch مربوط به میکروکنترلر
۳۸	۷.۲۱ تولید داده در سنسورها
۳۸	۸.۲۱ قالب داده و checksum
۳۸	۹.۲۱ فایل main بخش سوم
۳۹	۱۰.۲۱ اجزئیات پیاده سازی بخش سوم
۳۹	۱۰.۲۱.۱ فایل I2CMux.h: مدل سخت افزاری مالتی پلکسر
۴۳	۱۰.۲۱.۲ فایل I2CMultiplexer.h: رفتار کاربردی مالتی پلکسر
۴۵	۱۰.۲۱.۳ فایل MuxMicrocontroller.h: رفتار میکروکنترلر در تست I2C MUX
۴۷	۱۱.۲۱.۱ فایل part3_main.cpp: اجرای مستقل سناریوی I2C MUX
۴۸	۲.۲۱ خروجی کامل تست مستقل بخش سوم
۵۲	۳.۲۱ ابررسی خروجی اجرای بخش سوم
۵۲	۴.۲۱ تحلیل تراکنش کانال صفر
۵۳	۵.۲۱ تحلیل تراکنش کانال یک
۵۴	۶.۲۱ تکرار انتخاب کانال صفر
۵۴	۷.۲۱ خلاصه نتایج خروجی بخش سوم
۵۵	۸.۲۱ نتیجه گیری از خروجی
۵۵	۲۲ پاسخ سوالات بخش اول: پروتکل آدرس محور C2I/I2C
۵۵	۱.۲۲ سوال اول: مشکل اتصال چند سنسور با آدرس مشابه روی یک باس
۵۶	۲.۲۲ سوال دوم: چهار اشکال نسخه ساده شده نسبت به C2I/I2C واقعی
۵۶	۳.۲۲ سوال سوم: کارکرد ByteStream، ByteVector و getByte
۵۷	۲۳ پاسخ سوالات بخش دوم: پروتکل USART Full Duplex
۵۷	۱.۲۳ سوال اول: چرا بیت های شروع و پایان با بیت های داده اشتباه گرفته نمی شوند؟
۵۷	۲.۲۳ سوال دوم: با نبود خط کلاک، USART چگونه داده ها را تشخیص می دهد؟
۵۸	۳.۲۳ سوال سوم: آیا پیاده سازی نرم افزاری USART با پین های دیجیتال ممکن است؟
۵۸	۲۴ پاسخ سوالات بخش سوم
۵۸	۱.۲۴ سوال اول: تابع reverse در Byte.h چه کاری انجام می دهد و آیا در بوردهای واقعی هم چنین عملیاتی استفاده می شود؟
۵۹	۲.۲۴ سوال دوم: توابع takeNthBit در Bit.h چه کاری انجام می دهند و تفاوت آن ها چیست؟
۶۰	۳.۲۴ سوال سوم: تابع attachPinToCommunicationClock از کلاس Board چه کاری انجام می دهد؟
۶۰	۲۵ تصمیم های فنی مهم
۶۰	۱.۲۵ استفاده از ماشین حالت
۶۰	۲.۲۵ جدا کردن لایه کاربردی از لایه پروتکل
۶۰	۳.۲۵ استفاده از صف و بافر
۶۱	۴.۲۵ اعتبارسنجی داده سنسور

۶۱	۲۶ چالش های پیاده سازی
۶۱	۱.۲۶ هماهنگی بین Slave و Master
۶۱	۲.۲۶ ترتیب بایت ها در داده ۱۶ بیتی
۶۱	۳.۲۶ اتصال درست RX و TX
۶۱	۴.۲۶ اجرای همزمان دو پروتکل روی میکروکنترلر
۶۱	۲۷ فایل های استفاده شده در پروژه
۶۱	۱.۲۷ فایل های آماده و زیرساختی
۶۱	۲.۲۷ فایل های اصلی پیاده سازی پروژه
۶۳	۲۸ تقسیم کار اعضای گروه
۶۳	۲۹ جمع بندی

۱ مقدمه

در این تمرین هدف اصلی ما توسعه یک بستر شبیه سازی برای سامانه های نهفته و اضافه کردن رفتار ارتباطی به آن بود. در سامانه های نهفته، اتصال دو قطعه فقط به معنی اتصال چند پایه فیزیکی نیست. برای مثال، وقتی یک میکروکنترلر به یک سنسور متصل می شود، علاوه بر پایه های تغذیه و زمین، باید یک قرارداد ارتباطی مشخص نیز وجود داشته باشد تا دو طرف بدانند چه زمانی باید داده ارسال کنند، چه زمانی باید داده بخوانند، داده چگونه بسته بندی می شود، آدرس دهی چگونه انجام می شود و دریافت داده چگونه اعتبارسنجی می شود. این قرارداد همان پروتکل ارتباطی است.

در پروژه حاضر، یک چارچوب شبیه سازی در اختیار ما قرار گرفت که امکان مدل کردن بوردها، پردازنده، پایه های GPIO، لینک های فیزیکی، واحدهای پایه مانند بیت و بایت، ولتاژ، فرکانس و همچنین ساختار Sketch ها را فراهم می کرد. این چارچوب به ما اجازه می داد قطعات مختلف را مانند دنیای واقعی از طریق پایه ها و لینک ها به هم متصل کنیم. با این حال، برای اینکه ارتباط بین قطعات معنی دار شود، لازم بود منطق پروتکل های ارتباطی روی این بستر تکمیل شود.

در این مرحله از پروژه، تمرکز ما روی سه بخش اصلی بود. بخش اول مربوط به پیاده سازی و تکمیل یک پروتکل آدرس محور شبیه C2I/I2C برای ارتباط بین میکروکنترلر ESP8266 و سنسور VL530X بود. بخش دوم مربوط به پیاده سازی و تکمیل یک نسخه ساده شده از پروتکل USART برای ارتباط دوطرفه بین میکروکنترلر و یک حافظه یا هارد دیسک فرضی بود که در کد با بورد Usb و Sketch مربوط به HardDisk مدل شد. بخش سوم نیز مربوط به طراحی و تست یک I2C MUX بود؛ در این بخش، دو سنسور با آدرس یکسان 0x29 روی دو کانال جداگانه قرار گرفتند و میکروکنترلر از طریق ارتباط USART کانال موردنظر را برای مالتی پلکسر مشخص کرد. سپس I2C MUX از طریق کانال انتخاب شده با سنسور مربوطه ارتباط I2C برقرار کرد و داده سنسور را دوباره از طریق USART به میکروکنترلر برگرداند.

۲ هدف کلی پروژه

هدف پروژه این بود که یک شبیه ساز ساده سامانه های نهفته از حالت اتصال صرفاً فیزیکی خارج شود و رفتار ارتباطی واقعی تری پیدا کند. در ابتدای کار، امکان تعریف بوردها و اتصال پایه ها به لینک ها وجود داشت، اما صرفاً اتصال فیزیکی برای شبیه سازی یک ارتباط کامل کافی نیست. برای مثال اگر پایه SDA یک سنسور به پایه SDA میکروکنترلر وصل شود، هنوز مشخص نیست چه کسی ابتدا داده می نویسد، چه زمانی آدرس ارسال می شود، چه زمانی سنسور باید پاسخ دهد و داده نهایی چگونه از چند بایت بازسازی می شود.

بنابراین هدف ما این بود که از زیرساخت آماده پروژه استفاده کنیم و روی آن، رفتار پروتکل های ارتباطی را تکمیل کنیم. در بخش I2C/C2I، میکروکنترلر باید بتواند آدرس سنسور مقصد را مشخص کند، سنسور در صورت تطابق آدرس پاسخ دهد و سپس داده سنسور در قالب چند بایت منتقل شود. در بخش USART نیز باید یک ارتباط سریال دوطرفه طراحی شود که در آن میکروکنترلر یک آدرس برای حافظه فرضی ارسال کند و حافظه داده متناظر با آن آدرس را بازگرداند.

در بخش سوم، هدف این بود که مشکل وجود چند سنسور با آدرس یکسان روی یک باس I2C را به کمک یک مالتی پلکسر حل کنیم. در این حالت، هر سنسور روی یک کانال جداگانه از I2C MUX قرار می گیرد و در هر لحظه فقط یک کانال فعال می شود. به این ترتیب، حتی اگر دو سنسور آدرس یکسان داشته باشند، تداخلی روی باس رخ نمی دهد، چون فقط سنسور کانال انتخاب شده در تراکنش شرکت می کند. در پیاده سازی ما، میکروکنترلر از طریق USART به I2C MUX فرمان می دهد که کدام کانال فعال شود و MUX پس از خواندن داده سنسور، نتیجه را از طریق همان ارتباط USART به میکروکنترلر برمی گرداند.

از نظر معماری، تلاش کردیم پیاده سازی به صورت ماژولار باقی بماند. یعنی کدهای مربوط به بورد، پروتکل، Sketch و اتصال های فیزیکی در جای مناسب خود قرار بگیرند. به طور خلاصه، بوردها مشخصات سخت افزاری و پروتکل های قابل پشتیبانی را معرفی می کنند، پروتکل ها مسئول منطق ارتباطی هستند، Sketch ها رفتار کاربردی هر قطعه را مدل می کنند و تابع main اجزای سیستم را می سازد و به هم متصل می کند.

۳ معماری کلی شبیه ساز و کدهای آماده پروژه

چارچوب اصلی شبیه ساز به صورت آماده در اختیار ما قرار گرفته بود. این چارچوب شامل کلاس هایی برای مدل کردن بوردها، پردازنده، پایه های GPIO، لینک های ارتباطی، واحدهای بیت و بایت، ولتاژ، فرکانس، صف ها و Sketch ها بود. ما از این زیرساخت آماده برای پیاده سازی منطق پروتکل ها و تکمیل رفتار اجزای مورد نیاز تمرین استفاده کردیم.

۱.۳ کلاس Object و نام گذاری اجزا

کلاس Object یکی از کلاس های زیرساختی آماده پروژه بود. نقش این کلاس فراهم کردن امکان نام گذاری برای اجزای مختلف شبیه ساز است. در چنین پروژه ای، اجزایی مانند بورد، پردازنده، پایه و لینک زیاد می شوند و اگر هر شیء نام مشخصی نداشته باشد، دنبال کردن خروجی ها و پیام های دیباگ سخت می شود. در تابع main از ماکروه های آماده مانند CPS_SET_OBJECT_NAME_PTR و CPS_SET_OBJECT_NAME_PTR استفاده شد تا برای بوردها و لینک ها نام خوانا تنظیم شود. این کار به ما کمک کرد هنگام بررسی خروجی برنامه متوجه شویم هر پیام مربوط به کدام قطعه یا کدام لینک است.

۲.۳ کلاس های Pin و Link

فایل های Pin.h و Link.h نیز از قبل در اختیار ما قرار داشتند. این دو فایل بخش فیزیکی شبیه ساز را مدل می کنند. به صورت کلی، Pin نماینده یک پایه روی بورد است و Link نقش سیم یا اتصال بین پایه ها را دارد. وقتی یک پایه به یک لینک متصل می شود، خواندن و نوشتن روی آن پایه از طریق همان لینک با قطعه دیگر به اشتراک گذاشته می شود.

ما در این تمرین این کدها را از ابتدا پیاده سازی نکردیم، بلکه از آن ها برای ساختن اتصال های مورد نیاز بین ESP8266، VL530X و Usb استفاده کردیم. برای مثال، در ارتباط I2C/C2I، پایه های VDD، GND، SDA و SCL میکروکنترلر و سنسور به لینک های متناظر وصل شدند. در ارتباط USART نیز پایه TX یک طرف به RX طرف دیگر و پایه RX آن به TX طرف مقابل متصل شد.

۳.۳ کلاس Board

کلاس Board یکی دیگر از کلاس های آماده پروژه بود که مدل کلی بوردهای سخت افزاری را فراهم می کرد. این کلاس به صورت template نوشته شده و پارامترهایی مانند BaudRate، BitRate، Frequency، WorkingVoltage و نوع GPIO را دریافت می کند. با این طراحی، هر بورد می تواند مشخصات مخصوص خودش را داشته باشد و در عین حال از رفتار عمومی بوردها استفاده کند. هر بورد یک پردازنده داخلی و یک ساختار GPIO دارد. در چارچوب آماده پروژه، بورد مسئول نگهداری این اجزا و شروع اجرای پردازنده است. اگر فرکانس بورد مستقل باشد، پردازنده آن می تواند در یک thread جدا اجرا شود. اگر فرکانس بورد از نوع Driven باشد، بورد به clock خارجی وابسته است. این موضوع برای سنسور VL530X مهم بود، چون این سنسور باید با clock میکروکنترلر کار کند.

۴.۳ کلاس Processor

کلاس Processor نیز در کد پایه پروژه وجود داشت و نقش CPU هر بورد را در شبیه ساز ایفا می کرد. پردازنده مسئول اجرای توابع setup و loop، جلو بردن سیکل های اجرایی، drive کردن پایه های GPIO و اجرای پروتکل های نصب شده روی بورد است.

ما منطق داخلی Processor را به صورت جزئی تغییر ندادیم، بلکه از قابلیت آماده آن برای نصب و اجرای چند پروتکل استفاده کردیم. برای مثال، روی پردازنده ESP8266 هر دو پروتکل I2C و USART نصب شدند تا میکروکنترلر بتواند همزمان با سنسور و هارد دیسک فرضی ارتباط داشته باشد.

۵.۳ کلاس های پایه پروتکل

فایل Protocol.h و کلاس های پایه مرتبط با پروتکل ها نیز بخشی از زیرساخت آماده بودند. این فایل یک ساختار کلی برای پروتکل ها فراهم می کند. به طور کلی، هر پروتکل باید بتواند در هر سیکل اجرا شود، داده بنویسد، داده بخواند و وضعیت وجود داده آماده را اعلام کند. ساختار عمومی چنین API ای به صورت زیر است:

```
1 virtual void run(Gpio& gpio) = 0;
2 virtual void write(Byte byte) = 0;
3 virtual Byte read() = 0;
4 bool isDataAvailable() const;
```

کلاس پایه، یک buffer داخلی برای داده های دریافت شده فراهم می کند. ما از همین ساختار آماده استفاده کردیم و منطق اختصاصی دو پروتکل I2C/C2I و USART را در کلاس های مربوط به خودشان تکمیل کردیم.

۶.۳ کلاس AbstractSketch

فایل AbstractSketch.h نیز از کدهای آماده پروژه بود. این فایل ساختاری شبیه برنامه نویسی Arduino فراهم می کند. در این مدل، هر Sketch دو تابع اصلی دارد:

```
1 setup(Gpio& gpio)
2 loop(Gpio& gpio)
```

تابع setup یک بار در ابتدای اجرای بورد اجرا می شود و تابع loop به صورت تکراری در هر سیکل اجرا می شود. ما از این ساختار آماده استفاده کردیم و رفتار کاربردی سنسور، میکروکنترلر و هارددیسک فرضی را در Sketch های مربوط به آن ها تکمیل کردیم.

۷.۳ ابزارهای کمکی آماده

فایل هایی مانند Queue.h، ByteStream.h، Wave.h و فایل های موجود در بخش Units شامل Bit.h، Byte.h، BaudRate.h، Frequency.h، Voltage.h نیز از قبل آماده بودند. این فایل ها امکاناتی برای مدیریت صف ها، ساخت مقدار چندبایتی، استخراج بیت ها و بایت ها، مدل کردن ولتاژ، فرکانس و نرخ ارسال فراهم می کردند. ما این ابزارها را از ابتدا پیاده سازی نکردیم، بلکه از آن ها در پیاده سازی استفاده کردیم. برای مثال، در بخش سنسور از توابع مرتبط با بایت برای جدا کردن مقدار uint16_t به دو بایت استفاده شد و در سمت میکروکنترلر از ByteStream برای بازسازی مقدار چندبایتی کمک گرفتیم.

۴ تکمیل و استفاده از بورد ESP8266

بورد ESP8266 به عنوان یکی از بوردهای آماده پروژه در اختیار ما قرار داشت. این بورد شامل GPIO های اصلی مانند VDD، GND، TX، RX، SDA و SCL بود و می توانست نقش میکروکنترلر اصلی سیستم را بازی کند. در پیاده سازی ما، این بورد به عنوان نقطه مرکزی ارتباطات سیستم استفاده شد. از یک طرف باید با سنسور VL530X از طریق I2C/C2I ارتباط برقرار می کرد و از طرف دیگر باید با هارددیسک فرضی از طریق USART داده رد و بدل می کرد.

تغییر اصلی مربوط به بخش USART در ESP8266 این بود که پروتکل USART به صورت یک عضو mutable در بورد قرار داده شد و روی پردازنده نصب شد. همچنین یک تابع getter برای دسترسی به آن نوشته شد:

```
1 mutable Protocols::Usart<Esp8266, Gpio> m_usart {this};
2
3 inline Protocols::Usart<Esp8266, Gpio>&
4 usart() const
```



```

5 {
6     return m_usart;
7 }

```

به کمک این getter، Sketch میکروکنترلر می تواند با node()->usart() به پروتکل USART دسترسی داشته باشد و تابع init، write و read آن را صدا بزند. پایه های RX و TX برد از قبل در ساختار GPIO وجود داشتند و در main به برد Usb متصل شدند.

۵ تکمیل و استفاده از سنسور VL530X

سنسور VL530X نیز در کدهای اولیه پروژه به عنوان یک سنسور دارای پایه های VDD، GND، SDA و SCL معرفی شده بود. ما از این ساختار آماده استفاده کردیم و رفتار ارتباطی سنسور را از طریق نصب و استفاده از پروتکل I2CSlave تکمیل کردیم.

آدرس ثابت سنسور برابر 0x29 در نظر گرفته شد تا میکروکنترلر بتواند از طریق پروتکل آدرس محور با آن ارتباط برقرار کند. این آدرس در سمت master برای ارسال درخواست خواندن استفاده می شود و در سمت slave برای تشخیص اینکه درخواست مربوط به همین سنسور است یا نه. از نظر زمان بندی، سنسور مانند یک برد وابسته به clock در نظر گرفته شد. یعنی برخلاف میکروکنترلر، خودش clock مستقل برای ارتباط ندارد و با clock تولید شده توسط ESP8266 جلو می رود. این مدل با رفتار کلی سنسورهای I2C سازگار است، زیرا در ارتباط I2C معمولاً master خط SCL را کنترل می کند.

۶ پیاده سازی پروتکل I2C/C2I

۱.۶ هدف از پیاده سازی

در بخش اول پروژه، هدف ما پیاده سازی یک پروتکل آدرس محور شبیه I2C/C2I بود. در این ارتباط، میکروکنترلر ابتدا باید آدرس سنسور مقصد را مشخص کند. اگر سنسور آدرس خودش را تشخیص دهد، یک پاسخ تایید یا ACK می دهد و سپس داده آماده را برای میکروکنترلر ارسال می کند. برای این بخش، دو نقش اصلی وجود داشت: master و slave. ESP8266 نقش master را بر عهده داشت و سنسور VL530X نقش slave را. به همین دلیل، در کد پروتکل دو کلاس اصلی برای سمت master و سمت slave در نظر گرفته شد:

- I2CMaster

- I2CSlave

هر دو کلاس از ساختار پایه آماده پروتکل استفاده می کنند، اما منطق اختصاصی خودشان را برای ارسال آدرس، دریافت ACK، خواندن داده و ارسال داده دارند.

۲.۶ ساختار کلی ارتباط

در پیاده سازی انجام شده، ارتباط I2C/C2I به صورت یک تراکنش مرحله ای مدل شد. ابتدا master درخواست خواندن از آدرس مشخص را شروع می کند. سپس آدرس روی ساختار ارتباطی قرار می گیرد و slave آن را بررسی می کند. اگر آدرس با آدرس سنسور برابر باشد، slave مقدار ACK را آماده می کند. بعد از دریافت ACK، master وارد مرحله خواندن داده می شود.

برای کنترل وضعیت مشترک ارتباط، یک ساختار مشترک برای bus در نظر گرفته شد. این ساختار وضعیت هایی مانند فعال بودن تراکنش، آماده بودن آدرس، آماده بودن ACK، آماده بودن داده، بایت داده و پایان تراکنش را نگهداری می کند. چون master و slave می توانند در threadهای جداگانه اجرا شوند، برای جلوگیری از ناسازگاری داده از mutex استفاده شد.

۳.۶ ماشین حالت سمت Master

برای سمت master، پروتکل به صورت ماشین حالت طراحی شد. حالت های اصلی آن شامل حالت بیکار، ارسال آدرس، انتظار برای ACK، خواندن داده، نوشتن داده و پایان تراکنش بود. این طراحی باعث شد ارتباط در چند مرحله کنترل شود و همه کارها در یک تابع blocking انجام نشود.

در حالت بیکار، master منتظر درخواست جدید می ماند. وقتی Sketch میکروکنترلر تابع requestFrom را صدا می زند، آدرس مقصد و تعداد بایت های مورد انتظار تنظیم می شود. سپس پروتکل وارد مرحله ارسال آدرس می شود. بعد از ارسال آدرس، master منتظر ACK می ماند. اگر ACK دریافت شود، وارد مرحله خواندن داده می شود و بایت های دریافتی را در buffer داخلی قرار می دهد. برای جلوگیری از گیر کردن سیستم، در مرحله انتظار برای ACK یک منطق محدودکننده نیز در نظر گرفته شد تا اگر پاسخی دریافت نشد، تراکنش برای همیشه معلق نماند.

۴.۶ ماشین حالت سمت Slave

در سمت slave نیز منطق پروتکل به شکل ماشین حالت پیاده سازی شد. slave ابتدا منتظر دریافت آدرس می ماند. وقتی آدرس آماده می شود، آن را بررسی می کند و اگر با آدرس خودش برابر باشد، ACK را فعال می کند. سپس اگر درخواست از نوع خواندن باشد، داده های موجود در صف ارسال را به ترتیب آماده می کند تا master بتواند آن ها را بخواند.

این ساختار باعث شد سنسور فقط زمانی پاسخ دهد که آدرس خودش را دریافت کرده باشد. به این ترتیب مفهوم آدرس محور بودن پروتکل در شبیه ساز نشان داده شد.

۷ تکمیل Sketch سنسور

Sketch اولیه سنسور در کدهای پروژه وجود داشت، اما رفتار اصلی آن در این تمرین تکمیل شد. وظیفه Sketch سنسور این بود که در سمت VL530X یک داده سنسوری تولید کند و آن را از طریق پروتکل I2C/C2I در اختیار میکروکنترلر قرار دهد.

۱.۷ تابع setup سنسور

در تابع setup سنسور، پروتکل I2CSlave با آدرس سنسور مقداردهی شد. به این ترتیب، سنسور بعد از شروع اجرا می داند که باید فقط به درخواست هایی پاسخ دهد که آدرس آن ها با آدرس خودش برابر است.

```
1 node()->i2c().init(Sensors::VL530X::address);
```

۲.۷ تابع loop سنسور

در تابع loop سنسور، ابتدا بررسی می شود که آیا هنوز داده ای در صف ارسال پروتکل وجود دارد یا نه. اگر صف خالی باشد، سنسور یک مقدار جدید تولید می کند. مقدار تولید شده یک عدد در بازه ۰ تا ۴۰۰۰ است. این مقدار از نظر اندازه در یک بایت جا نمی شود، بنابراین به صورت یک مقدار ۱۶ بیتی نگهداری می شود. سپس مقدار ۱۶ بیتی به دو بایت تقسیم می شود. برای این کار از توابع کمکی آماده مربوط به واحد بایت استفاده شد. بعد از جدا کردن دو بایت، یک checksum ساده محاسبه می شود. این checksum برابر اختلاف بایت بزرگ تر و بایت کوچک تر است و همیشه مقدار نامنفی دارد. در نهایت، سنسور سه بایت را در صف ارسال قرار می دهد: بایت پرارزش تر مقدار سنسور، بایت کم ارزش تر مقدار checksum. این سه بایت بعداً در پاسخ به درخواست master از طریق پروتکل I2C/C2I ارسال می شوند.

۳.۷ دلیل استفاده از checksum

استفاده از checksum باعث شد سمت میکروکنترلر بتواند صحت داده دریافتی را بررسی کند. اگر دو بایت داده به هر دلیلی با ترتیب اشتباه یا مقدار نامعتبر دریافت شوند، احتمال زیادی وجود دارد که checksum محاسبه شده با checksum دریافتی برابر نشود. در این حالت میکروکنترلر داده را نامعتبر تشخیص می دهد و آن را به عنوان خروجی معتبر گزارش نمی کند.

۸ تکمیل Sketch میکروکنترلر

Sketch اولیه میکروکنترلر در کدهای پروژه وجود داشت و در این تمرین تکمیل شد. این Sketch رفتار اصلی ESP8266 را مشخص می کند. در نسخه تکمیل شده، میکروکنترلر هم با سنسور از طریق I2C/C2I ارتباط برقرار می کند و هم از طریق USART با هارددیسک فرضی داده رد و بدل می کند.

۱.۸ تابع setup میکروکنترلر

در تابع setup، علاوه بر مقداردهی اولیه ارتباط I2C، پروتکل USART نیز مقداردهی شد. برای این کار، getter مربوط به USART از بورد ESP8266 صدا زده شد و سپس تابع init با gpio میکروکنترلر اجرا شد:

```
1 node()->i2c().init(0x29);
2 node()->usart().init(gpio);
```

خط اول، I2C را برای ارتباط با سنسور آدرس 0x29 آماده می کند. خط دوم، USART را برای استفاده از پایه های TX و RX میکروکنترلر آماده می کند.

۲.۸ تابع loop میکروکنترلر

در تابع loop، شمارنده m_cycleCount در هر سیکل افزایش پیدا می کند. سپس دو تابع جداگانه صدا زده می شوند:

```
1 handle_i2c();
2 handle_usart();
```

تابع handle_i2c ارتباط با سنسور را مدیریت می کند. تابع handle_usart برای بخش USART اضافه شد تا ارسال آدرس به هارددیسک و خواندن پاسخ آن را انجام دهد.

۳.۸ منطق تابع handle_usart

در تابع handle_usart، هر ۲۰۰ سیکل یک بار و تا زمانی که تعداد آدرس های ارسالی از حد مشخص بیشتر نشده باشد، یک آدرس جدید برای Usb/HardDisk ارسال می شود. آدرس ها از صفر شروع می شوند و در نسخه فعلی تا ۹ ادامه پیدا می کنند. برای همین یک متغیر addrs و یک مقدار محدودکننده limit_usart برابر ۱۰ در بخش خصوصی کلاس قرار داده شد.

```
1 if (m_cycleCount % 200 == 0 && addrs < limit_usart)
2 {
3     node()->usart().write(addrs);
4     addrs++;
5 }
```

بعد از ارسال آدرس، میکروکنترلر در هر سیکل بررسی می کند که آیا پاسخی از سمت USART آماده شده است یا نه. اگر داده ای وجود داشته باشد، آن را با read می خواند و در خروجی چاپ می کند.

۴.۸ همزمانی ارتباط I2C و USART

در نسخه نهایی این Sketch، دو ارتباط به صورت همزمان در loop مدیریت می شوند. از یک طرف، `handle_i2c` داده سنسور را دریافت و `checksum` آن را بررسی می کند. از طرف دیگر، `handle_usart` آدرس های حافظه را برای هارددیسک می فرستد و پاسخ را دریافت می کند. خروجی اجرای برنامه نیز نشان می دهد که هر دو ارتباط در کنار هم فعال هستند.

۹ مدیریت داده چندبایتی در ارتباط با سنسور

یکی از بخش های مهم پیاده سازی، انتقال مقدار سنسور بود. مقدار سنسور در بازه ۰ تا ۴۰۰۰ قرار دارد و این مقدار در یک بایت قابل ذخیره نیست. بنابراین لازم بود مقدار سنسور به دو بایت تقسیم شود و در سمت گیرنده دوباره بازسازی شود.

در سمت سنسور، مقدار تولید شده به صورت `uint16_t` نگهداری شد. سپس با استفاده از توابع آماده مربوط به بایت، دو بایت آن استخراج شد. بعد از آن، `checksum` محاسبه شد و سه بایت برای ارسال آماده شدند. در سمت میکروکنترلر، دو بایت اول با استفاده از `ByteStream` دوباره به یک مقدار ۱۶ بیتی تبدیل شدند. سپس `checksum` مورد انتظار محاسبه شد و با `checksum` دریافتی مقایسه شد. اگر دو مقدار برابر بودند، داده معتبر در نظر گرفته شد و مقدار نهایی چاپ شد.

۱۰ پیاده سازی پروتکل USART

۱.۱۰ هدف از پیاده سازی

در بخش دوم پروژه، هدف ما پیاده سازی یک نسخه ساده شده از پروتکل USART بود. USART یک ارتباط سریال دوطرفه است که در آن هر سمت می تواند هم ارسال کننده و هم دریافت کننده باشد. در این تمرین، میکروکنترلر باید یک آدرس را برای هارددیسک فرضی ارسال کند و هارددیسک نیز داده متناظر با همان آدرس را به میکروکنترلر برگرداند. بنابراین لازم بود مسیر ارسال و مسیر دریافت به صورت جداگانه و همزمان در پروتکل مدیریت شوند.

در ساختار کلی، USART از کلاس پایه `AbstractUsart` ارث بری می کند و همانند سایر پروتکل ها تابع `run` آن در هر سیکل پردازنده اجرا می شود. در هر اجرای `run`، دو بخش اصلی صدا زده می شوند: یکی برای دریافت داده و دیگری برای ارسال داده. به همین دلیل، منطق پروتکل به دو قسمت RX و TX تقسیم شد.

۲.۱۰ مقداردهی اولیه پروتکل

در تابع `init`، جهت منطقی پایه های RX و TX مشخص شد. در ارتباط USART، هر بورد باید از پایه RX بخواند و روی پایه TX بنویسد. بنابراین در مقداردهی اولیه، خواندن از TX غیرفعال و خواندن از RX فعال شد:

```
1 gpio.tx.setCanRead(false);  
2 gpio.rx.setCanRead(true);
```

این تابع هم در سمت میکروکنترلر و هم در سمت هارددیسک صدا زده می شود. تفاوتی که باعث درست کار کردن ارتباط می شود، در اتصال فیزیکی پایه ها در main است: TX میکروکنترلر به RX بورد وصل شده و TX بورد Usb به RX میکروکنترلر متصل شده است. بنابراین هر طرف همچنان از RX خودش می خواند و روی TX خودش می نویسد، اما داده در سمت مقابل از پایه درست دریافت می شود. بعد از تنظیم پایه ها، وضعیت ماشین حالت دریافت و ارسال روی حالت اولیه قرار داده می شود و پیام [USART] Initialized در خروجی چاپ می شود.

در تابع init پروتکل USART، خواندن از پایه TX غیرفعال و خواندن از پایه RX فعال شد. این کار باعث می شود هر بورد فقط از مسیر دریافت خودش داده بخواند و روی مسیر ارسال خودش داده بنویسد. اتصال ضربدری در main باعث می شود داده نوشته شده روی TX یک بورد، از طریق RX بورد مقابل قابل خواندن باشد.

۳.۱۰ ساختار فریم

در این پیاده سازی، هر بایت داده به صورت یک فریم ساده USART ارسال می شود. فریم شامل سه بخش اصلی است:

۱. Start Bit

۲. هشت بیت داده

۳. Stop Bit

خط ارتباطی در حالت عادی مقدار ۱ دارد. وقتی ارسال شروع می شود، ابتدا یک بیت صفر به عنوان start bit نوشته می شود. سپس هشت بیت داده به ترتیب از بیت کم ارزش تر ارسال می شوند. در پایان نیز یک بیت یک به عنوان stop bit روی خط نوشته می شود. در این تمرین، parity bit پیاده سازی نشد، چون در صورت تمرین نیز پیاده سازی آن لازم نبود.

۴.۱۰ ماشین حالت دریافت

برای دریافت داده، سه وضعیت اصلی در نظر گرفته شد: Idle، Data و Stop. در حالت Idle، پروتکل منتظر دریافت start bit است. اگر بیتی برای خواندن وجود نداشته باشد، تابع دریافت همان جا تمام می شود. اگر بیت خوانده شده صفر باشد، یعنی یک فریم جدید شروع شده است. در این حالت، بایت دریافت شونده و شمارنده بیت ها صفر می شوند و وضعیت به Data تغییر می کند. در حالت Data، بیت های دریافتی یکی یکی خوانده و داخل بایت فعلی قرار داده می شوند. چون ارسال داده به صورت LSB-first انجام می شود، هر بیت در جایگاه متناظر با m_rxBitIndex قرار می گیرد. وقتی هشت بیت دریافت شد، پروتکل وارد حالت Stop می شود. در حالت Stop، اگر بیت دریافتی برابر یک باشد، فریم معتبر تلقی می شود و بایت ساخته شده داخل m_buffer پروتکل قرار می گیرد. اگر stop bit مقدار درست نداشته باشد، داده معتبر در نظر گرفته نمی شود. در هر دو حالت، ماشین دریافت دوباره به Idle برمی گردد تا منتظر فریم بعدی بماند.

۵.۱۰ ماشین حالت ارسال

برای ارسال داده نیز چهار وضعیت اصلی در نظر گرفته شد: Idle، Start، Data و Stop. در حالت Idle، اگر صف ارسال خالی باشد، کاری انجام نمی شود. اما اگر write از بیرون صدا زده شده و داده ای در صف m_txQueue قرار گرفته باشد، اولین بایت از صف برداشته می شود و ماشین حالت وارد Start می شود. در حالت Start، یک بیت صفر روی TX نوشته می شود. در سیکل بعدی، وضعیت به Data می رود و بیت های داده با استفاده از تابع takeNthBit یکی یکی روی TX نوشته می شوند. بعد از ارسال هشت بیت، وضعیت به Stop تغییر می کند. در حالت Stop، یک بیت یک نوشته می شود و ماشین ارسال دوباره به Idle برمی گردد.

۶.۱۰ توابع read و write

در این پروتکل، تابع write مستقیماً بیت ها را روی خط نمی نویسد، بلکه فقط بایت موردنظر را وارد صف ارسال می کند:

```

1 void write(Byte byte) override
2 {
3     m_txQueue.push(byte);
4 }

```

بعد از آن، تابع run در سیکل های بعدی و از طریق ماشین حالت ارسال، این بایت را به فریم سریال تبدیل می کند. از طرف دیگر، تابع read داده را از m_buffer می خواند؛ یعنی از همان جایی که ماشین حالت دریافت، بایت های معتبر را قرار داده است. به این ترتیب، Sketch ها فقط با بایت کار می کنند و درگیر جزئیات start bit، بیت های داده و stop bit نمی شوند.

۱۱ پیاده سازی و استفاده از بورد Usb

برای بخش USART لازم بود یک بورد ساده داشته باشیم که فقط پایه های لازم برای ارتباط سریال را داشته باشد. این بورد با نام Usb تعریف شد و در سمت هارددیسک فرضی استفاده شد. از نظر ساختار، این بورد بسیار شبیه بوردهای آماده پروژه است، با این تفاوت که فقط پایه های موردنیاز ارتباط USART و تغذیه را دارد و دیگر از I2C پشتیبانی نمی کند.

GPIO این بورد شامل چهار پایه اصلی است: VDD، GND، RX و TX. مطابق راهنمای تمرین، BaudRate این بورد برابر NotSpecified قرار داده شد و BitRate نیز از همان مقدار ساخته شد. همچنین فرکانس بورد مستقل در نظر گرفته شد و از Frequency::F320khz استفاده شد، چون در صورت تمرین تاکید شده بود فرکانس کاری بورد مستقل باشد و روی Drived قرار نگیرد. در سازنده بورد Usb، پروتکل USART روی پردازنده نصب شد:

```

1 m_processor->installProtocol(&m_usart);

```

همچنین مانند ESP8266، یک عضو mutable از نوع Protocols::Usart در بورد قرار داده شد و یک تابع getter به نام usart() نوشته شد تا Sketch هارددیسک بتواند به پروتکل دسترسی داشته باشد. بنابراین نقش اصلی Usb این است که یک بورد ساده با چهار پایه و یک پروتکل USART نصب شده فراهم کند. در بورد Usb، مطابق صورت تمرین، مقدار BaudRate برابر BaudRates::NotSpecified قرار داده شد و BitRate نیز با BaudRates::same(BaudRates::NotSpecified) ساخته شد. همچنین فرکانس بورد مستقل و برابر Frequency::F320khz در نظر گرفته شد و از Frequency::Drived استفاده نکردیم.

۱۲ تکمیل بخش ارتباطی HardDisk

فایل HardDisk.h به صورت نیمه آماده در اختیار ما قرار داشت و شامل ساختار کلی Sketch هارددیسک و حافظه داخلی آن بود. در پیاده سازی ما، بخش ارتباطی این Sketch تکمیل شد. هدف این بود که HardDisk بتواند از طریق USART آدرس ارسال شده از سمت میکروکنترلر را بخواند، مقدار متناظر با آن آدرس را از حافظه داخلی پیدا کند و پاسخ را دوباره از طریق USART برای میکروکنترلر ارسال کند.

در این طراحی، HardDisk نقش یک حافظه ساده read-only را دارد. یعنی میکروکنترلر فقط آدرس می فرستد و HardDisk فقط مقدار ذخیره شده در آن آدرس را برمی گرداند. همین رفتار برای تست مسیر رفت و برگشت USART کافی بود، چون هم مسیر ارسال از میکروکنترلر به HardDisk و هم مسیر برگشت از HardDisk به میکروکنترلر را درگیر می کند.

۱.۱۲ حافظه داخلی

حافظه داخلی HardDisk با یک نگاشت از آدرس های ۸ بیتی به داده های ۸ بیتی مدل شده است. در کد، این نگاشت با ساختاری مانند std::unordered_map<Byte, Byte> نگهداری می شود. کل این جدول برای تست

استفاده می شود، اما لازم نیست همه مقادیر آن در متن گزارش آورده شود. برای توضیح رفتار سیستم، چند نمونه از مقادیرهای اول کافی است.

برای نمونه، چند مقدار اول موجود در حافظه داخلی HardDisk در جدول زیر آمده است. همین مقادیر در خروجی اجرای برنامه نیز دیده می شوند؛ مثلاً وقتی میکروکنترلر آدرس 0x00 را ارسال می کند، مقدار 173 و وقتی آدرس 0x01 را ارسال می کند، مقدار 59 دریافت می شود.

جدول ۱: چند نمونه از نگاشت آدرس به داده در HardDisk

Address	Returned Data
0x00	0xAD = 173
0x01	0x3B = 59
0x02	0xF2 = 242
0x03	0x7E = 126
0x04	0xC1 = 193

۲.۱۲ تابع setup

در تابع setup، پروتکل USART مربوط به برد USB مقداردهی اولیه می شود. این کار باعث می شود HardDisk بتواند از پایه های RX و TX برای دریافت آدرس و ارسال پاسخ استفاده کند. نمونه کلی این بخش به صورت زیر است:

```
1 node()->usart().init(gpio);
```

بعد از مقداردهی اولیه، پیام HardDisk setup completed چاپ می شود تا در خروجی بتوانیم مطمئن شویم که Sketch هارددیسک آماده کار شده است.

۳.۱۲ تابع loop

در تابع loop، ابتدا بررسی می شود که آیا داده ای از طریق USART دریافت شده است یا نه. اگر داده ای آماده باشد، آن داده به عنوان آدرس خوانده می شود. سپس این آدرس در حافظه داخلی جستجو می شود. اگر مقدار متناظر پیدا شود، همان مقدار از طریق USART برای میکروکنترلر ارسال می شود. اگر آدرس در حافظه وجود نداشته باشد، مقدار پیش فرض 0xFF برگردانده می شود. نمونه کلی منطق این بخش به صورت زیر است:

```
1 if (node()->usart().isDataAvailable())
2 {
3     Byte address = node()->usart().read();
4     auto it = m_storage.find(address);
5
6     if (it != m_storage.end())
7         node()->usart().write(it->second);
8     else
9         node()->usart().write(0xFF);
10 }
```

این بخش باعث شد بتوانیم ارتباط دوطرفه USART را به صورت عملی تست کنیم؛ چون هم مسیر ارسال آدرس از میکروکنترلر به HardDisk و هم مسیر برگشت پاسخ از HardDisk به میکروکنترلر فعال شد.

۱۳ اتصال اجزای سیستم در تابع main

تابع main وظیفه ساختن اجزای سیستم، ساخت لینک ها، اتصال پایه ها و شروع اجرای Sketch ها را بر عهده دارد. در این بخش، علاوه بر اتصال قبلی بین ESP8266 و VL530X برای ارتباط I2C/C2I، اتصال های لازم برای ارتباط USART بین ESP8266 و Usb نیز اضافه شد.

۱.۱۳ ساخت بوردها

در ابتدا سه برد اصلی ساخته شدند:

```
1 Boards::Esp8266 esp8266;  
2 Sensors::VL530x vl530x;  
3 Sensors::Usb usb;
```

در این ساختار، ESP8266 نقش میکروکنترلر اصلی را دارد، VL530X سنسور متصل از طریق I2C/C2I است و Usb برد ارتباطی مربوط به هارددیسک فرضی است.

۲.۱۳ لینک های ارتباط I2C/C2I

برای ارتباط I2C/C2I چهار لینک ساخته شد: linkRed برای VDD، linkBlack برای GND، linkGreen برای SCL و linkYellow برای SDA. این لینک ها بین میکروکنترلر و سنسور به صورت نظیر به نظیر استفاده شدند.

۳.۱۳ لینک های ارتباط USART

برای ارتباط USART چهار لینک جدید ساخته شد:

- linkTxEspRxUsb •
- linkRxEspTxUsb •
- linkRedUsb •
- linkBlackUsb •

دو لینک اول برای اتصال ضربدری TX و RX استفاده شدند. یعنی linkTxEspRxUsb به TX میکروکنترلر و RX برد Usb وصل شد تا داده ای که میکروکنترلر می نویسد، توسط Usb خوانده شود. در مقابل، linkRxEspTxUsb به RX میکروکنترلر و TX برد Usb وصل شد تا داده ای که Usb می نویسد، توسط میکروکنترلر خوانده شود. دو لینک دیگر یعنی linkRedUsb و linkBlackUsb برای VDD و GND برد Usb ساخته شدند. چون لینک های linkRed و linkBlack قبلاً برای اتصال ESP8266 و VL530X استفاده شده بودند، برای اتصال تغذیه و زمین Usb از لینک های جداگانه استفاده شد.

۴.۱۳ اتصال پایه ها

اتصال های اصلی USART در main به صورت زیر انجام شد:

```
1 esp8266.gpio().tx.attachLink(linkTxEspRxUsb);  
2 esp8266.gpio().rx.attachLink(linkRxEspTxUsb);  
3  
4 usb.gpio().tx.attachLink(linkRxEspTxUsb);  
5 usb.gpio().rx.attachLink(linkTxEspRxUsb);
```

این اتصال ضربدری مهم ترین نکته ارتباط USART است. هر طرف روی TX خودش می نویسد، اما طرف مقابل همان داده را از RX خودش می خواند.

۵.۱۳ شروع اجرای Sketch ها

بعد از ساخت بوردها و اتصال لینک ها، سه Sketch ساخته و اجرا شدند:

```
1 MicroController micro(&esp8266);  
2 Sensor disSen(&vl530x);  
3 HardDisk hddisk(&usb);  
4  
5 micro.start();  
6 disSen.start();  
7 hddisk.start();
```

به این ترتیب، میکروکنترلر، سنسور و هارددیسک فرضی همزمان وارد اجرا شدند و در پایان نیز با Application::exec() برنامه زنده نگه داشته شد.

۱۴ جریان کلی اجرای برنامه

بعد از اجرای برنامه، ابتدا بوردها ساخته می شوند و پردازنده های داخلی آن ها آماده می شوند. سپس پروتکل های لازم روی پردازنده ها نصب می شوند. بعد از آن، لینک های فیزیکی ساخته شده و پایه های متناظر به آن ها وصل می شوند. سپس Sketch های میکروکنترلر، سنسور و هارددیسک شروع به کار می کنند.

در مرحله بعد، تابع setup هر Sketch فقط یک بار اجرا می شود. در این مرحله، سنسور آدرس I2C خود را تنظیم می کند، میکروکنترلر پروتکل های I2C و USART را آماده می کند و هارددیسک نیز USART خود را مقداردهی می کند.

بعد از پایان setup، تابع loop هر قطعه به صورت تکراری اجرا می شود. میکروکنترلر در بازه های زمانی مشخص از سنسور درخواست داده می کند و همچنین به صورت دوره ای آدرس هایی را برای هارددیسک فرضی می فرستد. سنسور داده تولید می کند و در صف ارسال I2C قرار می دهد. هارددیسک نیز آدرس های دریافتی از USART را پردازش می کند و پاسخ متناظر را برمی گرداند.

به این ترتیب، در نسخه فعلی برنامه دو ارتباط به صورت همزمان در حال کار هستند: ارتباط آدرس محور با سنسور و ارتباط سریال با هارددیسک فرضی.

۱۵ مدیریت زمان و سیکل ها

در Sketch میکروکنترلر از یک شمارنده سیکل استفاده شد. این شمارنده در هر بار اجرای loop افزایش پیدا می کند و برای کنترل زمان ارسال درخواست ها به کار می رود. هدف این بود که میکروکنترلر در هر سیکل درخواست جدید ارسال نکند، چون ممکن است تراکنش قبلی هنوز کامل نشده باشد.

برای ارتباط USART، هر ۲۰۰ سیکل یک آدرس جدید ارسال می شود و این کار فقط تا ۱۰ آدرس اول انجام می شود. برای ارتباط I2C/C2I نیز درخواست خواندن داده از سنسور به صورت دوره ای انجام می شود. این زمان بندی ساده باعث شد خروجی قابل دنبال کردن باشد و دو ارتباط بدون ایجاد فشار غیرضروری روی شبیه ساز در کنار هم اجرا شوند.

۱۶ طراحی خطایابی و پیام های خروجی

برای بررسی بهتر رفتار برنامه، در بخش های مختلف پیام های خروجی و debug قرار داده شد. این پیام ها کمک می کنند ترتیب اتفاقات را ببینیم و متوجه شویم ارتباط ها در کدام مرحله هستند.

۱.۱۶ پیام های راه اندازی

در ابتدای اجرا، پیام هایی برای تکمیل setup هر قطعه چاپ می شود. این پیام ها نشان می دهند که میکروکنترلر، سنسور و هارددیسک فرضی با موفقیت آماده شده اند.

۲.۱۶ پیام های ارتباط I2C/C2I

در ارتباط I2C/C2I، پیام هایی برای ارسال آدرس، دریافت ACK، ارسال داده و دریافت داده چاپ می شود. این پیام ها برای فهمیدن ترتیب تراکنش و بررسی درست بودن ارتباط master/slave مفید هستند.

۳.۱۶ پیام های ارتباط USART

در ارتباط USART، میکروکنترلر هنگام ارسال آدرس و دریافت پاسخ پیام چاپ می کند. هارددیسک نیز هنگام دریافت آدرس و ارسال داده متناظر پیام چاپ می کند. با کنار هم قرار دادن این پیام ها می توان مسیر رفت و برگشت داده را دنبال کرد.

۱۷ سناریوهای تست و خروجی اجرا

۱.۱۷ تست ارتباط I2C/C2I

برای تست ارتباط I2C/C2I، سناریوی زیر انجام شد:

۱. برد ESP8266 و سنسور VL530X ساخته شدند.
۲. پایه های VDD، GND، SCL و SDA به صورت نظیر به نظیر متصل شدند.
۳. سنسور با آدرس 0x29 مقداردهی شد.
۴. میکروکنترلر به صورت دوره ای از آدرس 0x29 سه بایت درخواست کرد.
۵. سنسور مقدار تصادفی تولید کرد و آن را در قالب دو بایت داده و یک بایت checksum ارسال کرد.
۶. میکروکنترلر دو بایت داده را بازسازی کرد.
۷. checksum بررسی شد.
۸. در صورت معتبر بودن داده، مقدار سنسور در خروجی چاپ شد.

۲.۱۷ تست ارتباط USART

برای تست ارتباط USART، سناریوی زیر انجام شد:

۱. برد ESP8266 و برد Usb ساخته شدند.
۲. پایه TX میکروکنترلر به پایه RX برد Usb متصل شد.
۳. پایه RX میکروکنترلر به پایه TX برد Usb متصل شد.
۴. میکروکنترلر به صورت دوره ای یک آدرس ۸ بیتی ارسال کرد.
۵. HardDisk آدرس را از طریق USART دریافت کرد.

۶. مقدار متناظر با آدرس از حافظه داخلی خوانده شد.

۷. پاسخ از طریق USART به میکروکنترلر برگشت داده شد.

۸. میکروکنترلر مقدار دریافتی را چاپ کرد.

۳.۱۷ نمونه خروجی اجرا

در خروجی اجرای برنامه، ابتدا ساخته شدن بوردها و اتصال لینک ها دیده می شود. برای مثال، پیام های مربوط به ساخته شدن ESP8266، VL530X و Usb چاپ می شوند و سپس اتصال پایه های مختلف به لینک ها گزارش می شود. بعد از آن، پیام های مقداردهی اولیه پروتکل ها و setup مربوط به MicroController، Sensor، HardDisk دیده می شود.

نمونه ای از بخش مهم خروجی به صورت زیر است:

```
Usb::processor created.
one instance of Usb created.
[USART] Initialized
HardDisk setup completed.
[USART] Initialized
esp8266 setup completed.
[I2C Slave] Initialized - own address 0x29
vl530x setup completed.

[Microcontroller] USART Write to USB: 0
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 5 sent
[I2C Master] Read data byte: 5
[I2C Slave] Data byte 218 sent
[I2C Master] Read data byte: 218
[I2C Slave] Data byte 213 sent
[I2C Master] Read data byte: 213
[HardDisk] USART Read from Micro address: 0
[HardDisk] USART Write to Micro data: 173
[Microcontroller] new distance: 1498
[Microcontroller] USART Read from USB: 173

[Microcontroller] USART Write to USB: 1
[HardDisk] USART Read from Micro address: 1
[HardDisk] USART Write to Micro data: 59
[Microcontroller] new distance: 3186
[Microcontroller] USART Read from USB: 59
```

این خروجی نشان می دهد که هر دو ارتباط همزمان فعال هستند. در بخش I2C/C2I، میکروکنترلر آدرس 0x29 را در حالت خواندن ارسال می کند، سنسور آدرس را تشخیص می دهد، ACK می فرستد و سپس سه بایت داده را ارسال می کند. در ادامه، میکروکنترلر مقدار نهایی سنسور را پس از بررسی checksum چاپ می کند. در بخش USART نیز میکروکنترلر ابتدا آدرس صفر را برای Usb/HardDisk ارسال می کند. سپس HardDisk پیام USART Read from Micro address: 0 را چاپ می کند، مقدار متناظر با این آدرس یعنی ۱۷۳ را برمی گرداند و در نهایت میکروکنترلر همان مقدار را با پیام USART Read from USB: 173 دریافت

می کند. همین روند برای آدرس های بعدی نیز ادامه پیدا می کند؛ مثلاً برای آدرس ۱، مقدار ۵۹ برگردانده شده است.

بنابراین خروجی نشان می دهد که مسیر رفت و برگشت USART درست کار می کند و در کنار آن ارتباط I2C/C2I نیز همچنان فعال است.

سناریوی تست ما به صورت request/response طراحی شده است، اما ساختار پروتکل از نظر پیاده سازی قابلیت full-duplex دارد؛ چون مسیر دریافت و ارسال با دو ماشین حالت مستقل RX و TX مدیریت می شوند و هر دو در هر اجرای تابع run بررسی می شوند.

۱۸ خروجی اختصاصی تست I2C/C2I و تحلیل آن

علاوه بر خروجی نهایی که اجرای همزمان I2C/C2I و USART را نشان می دهد، یک خروجی جداگانه نیز برای بررسی دقیق تر بخش I2C/C2I ثبت شد. هدف از این خروجی این است که رفتار ارتباطی بین میکروکنترلر ESP8266 و سنسور VL530X با تمرکز روی مسیر I2C/C2I بررسی شود. در این خروجی، معیار اصلی ما ساخته شدن میکروکنترلر و سنسور، اتصال پایه های VDD، GND، SCL و SDA، مقداردهی I2C Master و I2C Slave، ارسال آدرس، دریافت ACK، انتقال سه بایت داده و اعتبارسنجی مقدار سنسور است.

در ابتدای خروجی، پردازنده و نمونه های ESP8266 و VL530X ساخته می شوند. سپس لینک های فیزیکی بین پایه های متناظر متصل می شوند. در این بخش، لینک linkRed برای تغذیه، لینک linkBlack برای زمین، لینک linkGreen برای خط SCL و لینک linkYellow برای خط SDA استفاده شده است. پیام های attached successfully نشان می دهند که اتصال پایه ها به لینک ها درست انجام شده است.

بعد از اتصال لینک ها، پروتکل ها مقداردهی اولیه می شوند. در خروجی، I2C Master با آدرس هدف 0x29 مقداردهی می شود و I2C Slave نیز با آدرس داخلی 0x29 آماده می شود. این موضوع نشان می دهد که هر دو سمت ارتباط از یک آدرس مشترک استفاده می کنند و بنابراین سنسور باید بتواند درخواست میکروکنترلر را تشخیص دهد.

نکته مهم این است که در این تست، تمرکز تحلیل روی مسیر I2C/C2I است. اگر در خروجی پیام هایی مانند USART Initialized یا USART Write to USB دیده شوند، علت آن این است که همان Sketch میکروکنترلر اجرا شده است و تابع مربوط به USART نیز در loop صدا زده می شود. با این حال، در این بخش این پیام ها معیار تحلیل نیستند. معیار اصلی ما پیام های مربوط به ارسال آدرس، دریافت ACK، خواندن سه بایت داده و چاپ مقدار new distance است.

۱.۱۸ نحوه پیاده سازی تست اختصاصی

فایل part1_main.cpp سناریوی مستقل بخش اول را اجرا می کند. در این سناریو فقط یک میکروکنترلر ESP8266 و یک سنسور VL530X وجود دارد و ارتباط بین آن ها از طریق I2C/C2I انجام می شود. در ابتدا، دو بورد اصلی ساخته می شوند:

```
1 Boards::Esp8266 esp8266;
2 Sensors::VL530x vl530x;
```

سپس چهار لینک ساخته می شود:

```
1 auto linkRed      = std::make_shared<Link>();
2 auto linkBlack    = std::make_shared<Link>();
3 auto linkGreen    = std::make_shared<Link>();
4 auto linkYellow   = std::make_shared<Link>();
```

در این نام گذاری، linkRed برای تغذیه، linkBlack برای زمین، linkGreen برای SCL و linkYellow برای SDA استفاده شده است. سپس با ماکروهای CPS_SET_OBJECT_NAME_PTR و CPS_SET_OBJECT_NAME برای بوردها و لینک ها نام تنظیم می شود تا خروجی خوانا تر باشد. در مرحله بعد، پایه های میکروکنترلر به لینک ها وصل می شوند:

```

1 esp8266.gpio().vdd1.attachLink(linkRed);
2 esp8266.gpio().gnd1.attachLink(linkBlack);
3 esp8266.gpio().scl.attachLink(linkGreen);
4 esp8266.gpio().sda.attachLink(linkYellow);

```

و سنسور نیز به همان لینک ها متصل می شود:

```

1 vl530x.gpio().vdd.attachLink(linkRed);
2 vl530x.gpio().gnd.attachLink(linkBlack);
3 vl530x.gpio().scl.attachLink(linkGreen);
4 vl530x.gpio().sda.attachLink(linkYellow);

```

بنابراین میکروکنترلر و سنسور روی یک باس مشترک SCL/SDA قرار می گیرند. در انتها، Sketch های مربوط به میکروکنترلر و سنسور ساخته و شروع می شوند:

```

1 MicroController micro(&esp8266);
2 Sensor disSen(&vl530x);
3
4 micro.start();
5 disSen.start();

```

با اجرای Application::exec()، برنامه فعال می ماند و ارتباط دوره ای بین میکروکنترلر و سنسور انجام می شود.

۲۰۱۸ خروجی نهایی:

خروجی کامل این تست در فایل I2C_out.txt ذخیره شده است و با دستور زیر داخل گزارش آورده می شود. کافی است فایل I2C_out.txt کنار فایل اصلی HW.tex قرار داشته باشد:

```

Esp8266::Processor created.
one instance of Esp8266 created.
Vl530x::processor created.
one instance of Vl530x created.
linkRed: header Esp8266::vdd1 attached to link successfully.
Esp8266::vdd1: linkRed attached successfully
linkBlack: header Esp8266::gnd1 attached to link successfully.
Esp8266::gnd1: linkBlack attached successfully
linkGreen: header Esp8266::scl attached to link successfully.
Esp8266::scl: linkGreen attached successfully
linkYellow: header Esp8266::sda attached to link successfully.
Esp8266::sda: linkYellow attached successfully
linkRed: header Vl530x::vdd attached to link successfully.
Vl530x::vdd: linkRed attached successfully
linkBlack: header Vl530x::gnd attached to link successfully.
Vl530x::gnd: linkBlack attached successfully
linkGreen: header Vl530x::scl attached to link successfully.
Vl530x::scl: linkGreen attached successfully
linkYellow: header Vl530x::sda attached to link successfully.
Vl530x::sda: linkYellow attached successfully
[I2C Master] Initialized I2C target 0x29
[USART] Initialized
esp8266 setup completed.

```

```

[I2C Slave] Initialized I2C own address 0x29
v1530x setup completed.
[Microcontroller] USART Write to USB: 0
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 5 sent
[I2C Master] Read data byte: 5
[I2C Slave] Data byte 218 sent
[I2C Master] Read data byte: 218
[I2C Slave] Data byte 213 sent
[I2C Master] Read data byte: 213
[Microcontroller] new distance: 1498
[Microcontroller] USART Write to USB: 1
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 12 sent
[I2C Master] Read data byte: 12
[I2C Slave] Data byte 114 sent
[I2C Master] Read data byte: 114
[I2C Slave] Data byte 102 sent
[I2C Master] Read data byte: 102
[Microcontroller] new distance: 3186
[Microcontroller] USART Write to USB: 2
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 14 sent
[I2C Master] Read data byte: 14
[I2C Slave] Data byte 219 sent
[I2C Master] Read data byte: 219
[I2C Slave] Data byte 205 sent
[I2C Master] Read data byte: 205
[Microcontroller] new distance: 3803
[Microcontroller] USART Write to USB: 3
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 2 sent
[I2C Master] Read data byte: 2
[I2C Slave] Data byte 221 sent
[I2C Master] Read data byte: 221
[I2C Slave] Data byte 219 sent
[I2C Master] Read data byte: 219
[Microcontroller] new distance: 733
[Microcontroller] USART Write to USB: 4

```

```
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 11 sent
[I2C Master] Read data byte: 11
[I2C Slave] Data byte 112 sent
[I2C Master] Read data byte: 112
[I2C Slave] Data byte 101 sent
[I2C Master] Read data byte: 101
[Microcontroller] new distance: 2928
[Microcontroller] USART Write to USB: 5
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 12 sent
[I2C Master] Read data byte: 12
[I2C Slave] Data byte 47 sent
[I2C Master] Read data byte: 47
[I2C Slave] Data byte 35 sent
[I2C Master] Read data byte: 35
[Microcontroller] new distance: 3119
[Microcontroller] USART Write to USB: 6
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 9 sent
[I2C Master] Read data byte: 9
[I2C Slave] Data byte 91 sent
[I2C Master] Read data byte: 91
[I2C Slave] Data byte 82 sent
[I2C Master] Read data byte: 82
[Microcontroller] new distance: 2395
[Microcontroller] USART Write to USB: 7
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 9 sent
[I2C Master] Read data byte: 9
[I2C Slave] Data byte 83 sent
[I2C Master] Read data byte: 83
[I2C Slave] Data byte 74 sent
[I2C Master] Read data byte: 74
[Microcontroller] new distance: 2387
[Microcontroller] USART Write to USB: 8
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
```

```
[I2C Master] ACK received
[I2C Slave] Data byte 2 sent
[I2C Master] Read data byte: 2
[I2C Slave] Data byte 112 sent
[I2C Master] Read data byte: 112
[I2C Slave] Data byte 110 sent
[I2C Master] Read data byte: 110
[Microcontroller] new distance: 624
[Microcontroller] USART Write to USB: 9
```

در هر تراکنش I2C/C2I، ابتدا میکروکنترلر آدرس سنسور را در حالت خواندن ارسال می کند. آدرس اصلی سنسور 0x29 است، اما در خروجی مقدار 0x53 دیده می شود. دلیل این موضوع این است که در ارسال آدرس، بیت خواندن/نوشتن نیز در کنار آدرس قرار می گیرد. برای حالت خواندن، بیت آخر برابر ۱ است؛ بنابراین مقدار ارسالی به شکل زیر به دست می آید:

$$0x53 = (0x29 \ll 1) | 1$$

بعد از ارسال آدرس، سنسور پیام Address 0x29 R received را چاپ می کند. این پیام نشان می دهد که سنسور آدرس خودش را تشخیص داده و متوجه شده است که درخواست از نوع خواندن است. سپس سنسور ACK ارسال می کند و Master نیز دریافت ACK را گزارش می کند. پس از دریافت ACK، انتقال داده آغاز می شود. در هر بار خواندن، سنسور سه بایت ارسال می کند. دو بایت اول مقدار اصلی سنسور را تشکیل می دهند و بایت سوم برای checksum استفاده می شود. برای نمونه، در اولین تراکنش، سنسور سه بایت 5، 218 و 213 را ارسال کرده است. دو بایت اول مقدار اصلی سنسور را تشکیل می دهند و بایت سوم checksum است. مقدار نهایی چاپ شده توسط میکروکنترلر برابر 1498 است. همچنین اختلاف دو بایت 5 و 218 برابر 213 است، پس checksum درست است و داده معتبر تلقی شده است. در تراکنش دوم، سنسور بایت های 12، 114 و 102 را ارسال کرده و میکروکنترلر مقدار 3186 را چاپ کرده است. اختلاف 12 و 114 برابر 102 است، بنابراین checksum درست است. در تراکنش های بعدی نیز همین الگو تکرار شده است: آدرس ارسال می شود، ACK دریافت می شود، سه بایت داده منتقل می شوند و در نهایت مقدار distance فقط در صورت معتبر بودن checksum چاپ می شود. بنابراین این خروجی چند نکته مهم را تایید می کند. اول اینکه لینک های فیزیکی مربوط به I2C/C2I درست متصل شده اند. دوم اینکه Master و Slave هر دو با آدرس 0x29 مقداردهی شده اند. سوم اینکه سنسور فقط پس از تشخیص آدرس خودش ACK ارسال می کند. چهارم اینکه داده سنسور به صورت چندبایتی منتقل شده و میکروکنترلر آن را بازسازی می کند. پنجم اینکه checksum در سمت میکروکنترلر بررسی می شود و فقط در صورت معتبر بودن، مقدار نهایی با پیام new distance چاپ می شود.

۱۹ خروجی اختصاصی تست USART و تحلیل آن

برای بررسی دقیق تر بخش دوم تمرین، یعنی ارتباط USART بین میکروکنترلر و هارددیسک فرضی، یک خروجی جداگانه نیز ثبت شد. در این تست، برخلاف اجرای نهایی که هر دو ارتباط I2C/C2I و USART به صورت همزمان فعال بودند، تمرکز فقط روی مسیر USART است. به همین دلیل در این اجرا از یک نسخه اختصاصی تر از میکروکنترلر با نام UsartMicroController استفاده شده است که بخش I2C ندارد و فقط منطق ارسال آدرس و دریافت داده از طریق USART را اجرا می کند.

هدف از این تست این بود که مطمئن شویم مسیر ارتباطی USART به تنهایی و بدون دخالت بخش سنسور و I2C درست کار می کند. در این سناریو، میکروکنترلر چند آدرس متوالی را از طریق پایه TX خود ارسال می کند. بورد Usb این داده را از طریق پایه RX دریافت می کند و HardDisk مقدار متناظر با آن آدرس را از حافظه داخلی خود پیدا کرده و دوباره از طریق USART برای میکروکنترلر ارسال می کند.

۱.۱۹ ساخت بوردها و اتصال پایه ها

در ابتدای خروجی، ابتدا برد ESP8266 و برد Usb ساخته می شوند. در این تست، سنسور VL530X وجود ندارد، چون هدف فقط بررسی بخش USART است. پس از ساخته شدن بوردها، لینک های لازم برای تغذیه و ارتباط سریال متصل می شوند. دو لینک برای اتصال VDD و GND و دو لینک برای اتصال ضربدری TX/RX استفاده شده است.

اتصال ضربدری به این معنی است که TX میکروکنترلر به RX برد Usb وصل می شود و TX برد Usb نیز به RX میکروکنترلر وصل می شود. این اتصال برای ارتباط سریال ضروری است، چون هر طرف روی TX خودش می نویسد و طرف مقابل باید همان داده را از RX خودش بخواند.

۲.۱۹ مقداردهی اولیه USART

بعد از اتصال پایه ها، پروتکل USART در هر دو سمت مقداردهی اولیه می شود. یک بار این مقداردهی برای سمت میکروکنترلر انجام می شود و یک بار برای سمت Usb/HardDisk. به همین دلیل پیام [USART] Initialized دو بار در خروجی دیده می شود. پیام UsartMicroController setup completed نشان می دهد که میکروکنترلر مخصوص تست USART آماده شده است و پیام HardDisk setup completed نشان می دهد که HardDisk نیز آماده دریافت آدرس از سمت میکروکنترلر است.

در این تست، پیام های مربوط به I2C Master، I2C Slave، آدرس 0x29، ACK و داده سنسور دیده نمی شوند. این موضوع تایید می کند که خروجی حاضر برای تست مستقل بخش USART است.

۱.۲.۱۹ پیاده سازی تست

فایل part2_main.cpp سناریوی مستقل بخش دوم را اجرا می کند. در این سناریو، هدف فقط تست ارتباط USART بین میکروکنترلر و برد Usb/HardDisk است. در ابتدا، بوردهای لازم ساخته می شوند:

```
1 Boards::Esp8266 esp8266;  
2 Sensors::Usb usb;
```

سپس دو لینک برای تغذیه و زمین و دو لینک برای ارتباط سریال ساخته می شود:

```
1 auto linkVdd = std::make_shared<Link>();  
2 auto linkGnd = std::make_shared<Link>();  
3  
4 auto linkTxEspRxUsb = std::make_shared<Link>();  
5 auto linkRxEspTxUsb = std::make_shared<Link>();
```

در اینجا، ارتباط USART باید به صورت ضربدری وصل شود. بنابراین پایه های میکروکنترلر به شکل زیر وصل می شوند:

```
1 esp8266.gpio().tx.attachLink(linkTxEspRxUsb);  
2 esp8266.gpio().rx.attachLink(linkRxEspTxUsb);
```

و پایه های برد Usb به صورت معکوس به همان لینک ها متصل می شوند:

```
1 usb.gpio().tx.attachLink(linkRxEspTxUsb);  
2 usb.gpio().rx.attachLink(linkTxEspRxUsb);
```

این اتصال یعنی:

```
ESP8266::tx -> USB::rx  
USB::tx -> ESP8266::rx
```

پس هر طرف روی TX خودش می نویسد و طرف مقابل از RX خودش می خواند.
در پایان، Sketch های تست مستقل USART ساخته و اجرا می شوند:

```
1 UsartMicroController micro(&esp8266);
2 HardDisk                harddisk(&usb);
3
4 micro.start();
5 harddisk.start();
```

در این تست، UsartMicroController فقط آدرس ها را از طریق USART ارسال می کند و HardDisk مقدار متناظر با هر آدرس را برمی گرداند. چون سنسور و I2C در این فایل وجود ندارند، خروجی آن برای بررسی مستقل ارتباط USART مناسب است.

۳.۱۹ خروجی کامل تست USART

خروجی کامل این تست در فایل USART_out.txt ذخیره شده است. برای اینکه متن گزارش کامل باشد، خروجی کامل با دستور زیر در گزارش آورده می شود. کافی است فایل USART_out.txt کنار فایل اصلی HW.tex قرار داشته باشد:

```
D:\golboo\term 10\CPS\projects\A01_final\build>CPS4042_part2
Esp8266::Processor created.
one instance of Esp8266 created.
Usb::processor created.
one instance of Usb created.
linkVdd: header Esp8266::vdd1 attached to link successfully.
Esp8266::vdd1: linkVdd attached successfully
linkGnd: header Esp8266::gnd1 attached to link successfully.
Esp8266::gnd1: linkGnd attached successfully
linkTxEspRxUsb: header Esp8266::tx attached to link successfully.
Esp8266::tx: linkTxEspRxUsb attached successfully
linkRxEspTxUsb: header Esp8266::rx attached to link successfully.
Esp8266::rx: linkRxEspTxUsb attached successfully
linkVdd: header Usb::vdd attached to link successfully.
Usb::vdd: linkVdd attached successfully
linkGnd: header Usb::gnd attached to link successfully.
Usb::gnd: linkGnd attached successfully
linkRxEspTxUsb: header Usb::tx attached to link successfully.
Usb::tx: linkRxEspTxUsb attached successfully
linkTxEspRxUsb: header Usb::rx attached to link successfully.
Usb::rx: linkTxEspRxUsb attached successfully
[USART] Initialized
UsartMicroController setup completed.
[USART] Initialized
HardDisk setup completed.
[UsartMicroController] USART Write to USB address: 0
[HardDisk] USART Read from Micro address: 0
[HardDisk] USART Write to Micro data: 173
[UsartMicroController] USART Write to USB address: 1
[UsartMicroController] USART Read from USB data: 173
[HardDisk] USART Read from Micro address: 1
[HardDisk] USART Write to Micro data: 59[UsartMicroController] USART
Write to USB address: 2
```

```

[HardDisk] USART Read from Micro address: 2
[HardDisk] USART Write to Micro data: 242
[UsartMicroController] USART Write to USB address: 3
[UsartMicroController] USART Read from USB data: 59
[UsartMicroController] USART Read from USB data: 242
[HardDisk] USART Read from Micro address: 3
[HardDisk] USART Write to Micro data: 126
[UsartMicroController] USART Write to USB address: 4
[UsartMicroController] USART Read from USB data: 126
[HardDisk] USART Read from Micro address: 4
[HardDisk] USART Write to Micro data: 193
[UsartMicroController] USART Write to USB address: 5
[UsartMicroController] USART Read from USB data: 193
[HardDisk] USART Read from Micro address: 5
[HardDisk] USART Write to Micro data: [UsartMicroController] USART
Write to USB address: 6
69
[HardDisk] USART Read from Micro address: 6
[HardDisk] USART Write to Micro data: 152
[UsartMicroController] USART Write to USB address: 7
[UsartMicroController] USART Read from USB data: 69
[UsartMicroController] USART Read from USB data: 152
[HardDisk] USART Read from Micro address: 7
[HardDisk] USART Write to Micro data: 214
[UsartMicroController] USART Write to USB address: 8
[UsartMicroController] USART Read from USB data: 214
[HardDisk] USART Read from Micro address: 8
[HardDisk] USART Write to Micro data: 18
[UsartMicroController] USART Write to USB address: 9
[UsartMicroController] USART Read from USB data: 18
[HardDisk] USART Read from Micro address: 9
[HardDisk] USART Write to Micro data: 255
[UsartMicroController] USART Read from USB data: 255

```

۴.۱۹ تحلیل روند ارسال و دریافت آدرس ها

بعد از کامل شدن مرحله setup، میکروکنترلر شروع به ارسال آدرس ها از طریق USART می کند. ابتدا آدرس صفر ارسال می شود. سپس HardDisk همین آدرس را می خواند، مقدار متناظر با آن را از حافظه داخلی پیدا می کند و دوباره برای میکروکنترلر ارسال می کند. در خروجی، برای آدرس صفر مقدار 173 دریافت شده است. همین روند برای آدرس های بعدی نیز تکرار می شود. برای مثال، برای آدرس 1 مقدار 59، برای آدرس 2 مقدار 242، برای آدرس 3 مقدار 126، برای آدرس 4 مقدار 193 و برای آدرس 5 مقدار 69 برگشت داده شده است. این مقادیر مطابق نگاشت داخلی HardDisk هستند.

جدول ۲: نمونه آدرس های ارسال شده و داده های دریافت شده در تست مستقل USART

Address Sent by Microcontroller	Data Returned by HardDisk
0	173
1	59
2	242
3	126
4	193
5	69
6	152
7	214
8	18
9	255

این جدول نشان می دهد که HardDisk برای هر آدرس، مقدار متناظر را از حافظه داخلی خود پیدا کرده و از مسیر برگشت USART برای میکروکنترلر ارسال کرده است. بنابراین تست فقط ارسال یک طرفه نیست، بلکه مسیر رفت و برگشت داده را بررسی می کند.

۵.۱۹ نکته مربوط به ترتیب چاپ پیام ها

در برخی قسمت های خروجی، پیام های HardDisk و UsartMicroController ممکن است در کنار هم یا حتی روی یک خط چاپ شوند. دلیل این رفتار، اجرای همزمان اجزای شبیه ساز و استفاده مشترک از std::cout در چند thread مختلف است. یعنی چند بخش برنامه همزمان در حال چاپ پیام هستند و ممکن است خروجی های آن ها در ترمینال به صورت درهم یا بدون فاصله کامل دیده شود. این موضوع به معنی خطای منطقی در پروتکل نیست. معیار اصلی در این تست این است که همه آدرس های ارسال شده توسط میکروکنترلر در HardDisk خوانده شده اند و همه داده های متناظر در نهایت توسط میکروکنترلر دریافت شده اند. خروجی نشان می دهد که این شرط برقرار است.

۶.۱۹ جمع بندی تست مستقل USART

خروجی اختصاصی USART نشان می دهد که بخش دوم تمرین به درستی کار می کند. در این تست، بوردهای ESP8266 و Usb ساخته شده اند، پایه های VDD و GND متصل شده اند، پایه های TX و RX به صورت ضربدری وصل شده اند، پروتکل USART در هر دو سمت مقداردهی اولیه شده و سپس میکروکنترلر آدرس های 0 تا 9 را برای HardDisk ارسال کرده است. در سمت مقابل، HardDisk هر آدرس را خوانده، مقدار متناظر را از حافظه داخلی پیدا کرده و آن مقدار را به میکروکنترلر برگردانده است. میکروکنترلر نیز داده های برگشتی را از USB خوانده و در خروجی چاپ کرده است. بنابراین این تست تایید می کند که ارتباط دوطرفه USART، اتصال ضربدری TX/RX، ماشین حالت ارسال و دریافت، صف ارسال و بافر دریافت همگی به صورت مورد انتظار عمل می کنند.

۲۰ تحلیل خروجی اجرای برنامه در تست ترکیبی

در این بخش خروجی اجرای برنامه بررسی می شود. هدف از این خروجی این است که نشان دهیم هر دو ارتباط I2C/C2I و USART به صورت همزمان در شبیه ساز فعال هستند و میکروکنترلر می تواند هم از سنسور داده دریافت کند و هم از هارددیسک فرضی داده بخواند.

۱.۲۰ مرحله ساخت بوردها و اتصال لینک ها

در ابتدای خروجی، ابتدا پردازنده ها و بوردهای اصلی ساخته می شوند. پیام هایی مانند ساخته شدن Esp8266::Processor، ساخته شدن نمونه Esp8266، ساخته شدن V1530x و ساخته شدن Usb نشان می دهند که سه قطعه اصلی سیستم با موفقیت ایجاد شده اند.

بعد از آن، پیام های مربوط به اتصال لینک ها چاپ می شوند. در این بخش، پایه های VDD، GND، SCL و SDA بین ESP8266 و VL530X متصل می شوند. همچنین پایه های TX و RX بین ESP8266 و Usb به صورت ضربدری وصل می شوند. اتصال ضربدری یعنی TX میکروکنترلر به RX برد Usb و TX برد Usb به RX میکروکنترلر متصل شده است. این اتصال برای کارکرد صحیح USART ضروری است.

```
PS D:\UT\term 8\CPS\CA\CPS\A01\build> ".D:\UT\term 8\CPS\CA\CPS\A01\build\CPS4042.exe"
Esp8266::Processor created.
one instance of Esp8266 created.
V1530x::processor created.
one instance of V1530x created.
Usb::processor created.
one instance of Usb created.
linkRed: header Esp8266::vdd1 attached to link successfully.
Esp8266::vdd1: linkRed attached successfully
linkBlack: header Esp8266::gnd1 attached to link successfully.
Esp8266::gnd1: linkBlack attached successfully
linkGreen: header Esp8266::scl attached to link successfully.
Esp8266::scl: linkGreen attached successfully
linkYellow: header Esp8266::sda attached to link successfully.
Esp8266::sda: linkYellow attached successfully
linkTxEspRxUsb: header Esp8266::tx attached to link successfully.
Esp8266::tx: linkTxEspRxUsb attached successfully
linkRxEspTxUsb: header Esp8266::rx attached to link successfully.
Esp8266::rx: linkRxEspTxUsb attached successfully
linkRed: header V1530x::vdd attached to link successfully.
V1530x::vdd: linkRed attached successfully
linkBlack: header V1530x::gnd attached to link successfully.
V1530x::gnd: linkBlack attached successfully
linkGreen: header V1530x::scl attached to link successfully.
V1530x::scl: linkGreen attached successfully
linkYellow: header V1530x::sda attached to link successfully.
V1530x::sda: linkYellow attached successfully
linkRedUsb: header Usb::vdd attached to link successfully.
Usb::vdd: linkRedUsb attached successfully
linkBlackUsb: header Usb::gnd attached to link successfully.
Usb::gnd: linkBlackUsb attached successfully
linkRxEspTxUsb: header Usb::tx attached to link successfully.
Usb::tx: linkRxEspTxUsb attached successfully
linkTxEspRxUsb: header Usb::rx attached to link successfully.
```

شکل ۱: ساخته شدن بوردها و اتصال لینک های مربوط به I2C/C2I و USART

همان طور که در شکل ۱ دیده می شود، لینک های مربوط به I2C/C2I و USART با موفقیت به پایه های متناظر متصل شده اند. پیام های attached successfully نشان می دهند که اتصال های فیزیکی شبیه ساز درست انجام شده اند.

۲.۲۰ مرحله مقداردهی اولیه پروتکل ها

بعد از اتصال لینک ها، توابع setup مربوط به HardDisk، میکروکنترلر و سنسور اجرا می شوند. در این مرحله پیام [USART] Initialized برای مقداردهی اولیه پروتکل USART چاپ می شود. این پیام هم در سمت میکروکنترلر و هم در سمت HardDisk دیده می شود، چون هر دو طرف باید پروتکل USART خود را آماده کنند. همچنین پیام [I2C Slave] Initialized نشان می دهد که سنسور با آدرس 0x29 آماده شده است. در ادامه، پیام های setup completed نشان می دهند که هر سه Sketch اصلی با موفقیت راه اندازی شده اند.

```

PS D:\UT\term 8\CPS\CA\CPS\A01\build> ."D:\UT\term 8\CPS\CA\CPS\A01\build\CPS4042.exe"
Usb::tx: linkRxEspTxUsb attached successfully
linkTxEspRxUsb: header Usb::rx attached to link successfully.
Usb::rx: linkTxEspRxUsb attached successfully
[I2C Master] Initialized rco target 0x[USART] Initialized
HardDisk setup completed.
29
[USART] Initialized
esp8266 setup completed.
[I2C Slave] Initialized rco own address 0x29
v1530x setup completed.
[Microcontroller] USART Write to USB: 0
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 5 sent
[I2C Master] Read data byte: 5
[I2C Slave] Data byte 218 sent
[I2C Master] Read data byte: 218
[I2C Slave] Data byte 213 sent
[I2C Master] Read data byte: 213
[HardDisk] USART Read from Micro address: 0
[HardDisk] USART Write to Micro data: 173[Microcontroller] new distance: 1498

[Microcontroller] USART Read from USB: 173
[Microcontroller] USART Write to USB: 1
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 12 sent
[I2C Master] Read data byte: 12
[I2C Slave] Data byte 114 sent
[I2C Master] Read data byte: 114

```

شکل ۲: مقداردهی اولیه پروتکل ها و شروع اولین تراکنش های I2C و USART

در شکل ۲، بعد از آماده شدن سیستم، میکروکنترلر اولین آدرس را از طریق USART برای Usb/HardDisk ارسال می کند. پیام زیر نشان دهنده ارسال آدرس صفر است:

```
[Microcontroller] USART Write to USB: 0
```

در همین زمان، ارتباط I2C/C2I نیز شروع می شود. پیام زیر نشان می دهد که میکروکنترلر آدرس سنسور را در حالت خواندن روی باس ارسال کرده است:

```
[I2C Master] Address byte 0x53 (READ) sent on SDA
```

علت دیده شدن مقدار 0x53 این است که آدرس اصلی سنسور 0x29 است و در پروتکل آدرس محور، آدرس همراه با بیت خواندن/نوشتن ارسال می شود. در حالت خواندن، آدرس نهایی به صورت 0x53 دیده می شود. سپس سنسور آدرس 0x29 را تشخیص می دهد، پیام ACK ارسال می کند و سه بایت داده را برای میکروکنترلر می فرستد.

۳.۲۰ تحلیل همزمانی ارتباط I2C/C2I و USART

یکی از نکات مهم خروجی این است که پیام های مربوط به I2C/C2I و USART در کنار هم دیده می شوند. این موضوع نشان می دهد که میکروکنترلر در هر سیکل فقط یک نوع ارتباط را انجام نمی دهد، بلکه هر دو بخش `handle_usart` و `handle_i2c` در `loop` اجرا می شوند. برای مثال، در خروجی مربوط به آدرس های اولیه، ابتدا میکروکنترلر از طریق USART آدرس صفر را ارسال می کند. سپس HardDisk همان آدرس را دریافت می کند و مقدار متناظر با آن یعنی ۱۷۳ را برمی گرداند:

```

[HardDisk] USART Read from Micro address: 0
[HardDisk] USART Write to Micro data: 173
[Microcontroller] USART Read from USB: 173

```

این سه خط نشان می دهند که مسیر رفت و برگشت USART درست کار کرده است. ابتدا میکروکنترلر آدرس را نوشته، سپس HardDisk آن را خوانده و در نهایت مقدار متناظر دوباره توسط میکروکنترلر دریافت شده است. در همان بازه، پیام های I2C نیز دیده می شوند. سنسور سه بایت داده ارسال می کند و میکروکنترلر آن ها را می خواند. پس از دریافت کامل داده و بررسی checksum، مقدار سنسور با پیام new distance چاپ می شود.

```
PS D:\UT\term 8\CPS\CA\CPS\A01\build> . "D:\UT\term 8\CPS\CA\CPS\A01\build\CPS4042.exe"
[I2C Master] Read data byte: 12
[I2C Slave] Data byte 114 sent
[I2C Master] Read data byte: 114
[I2C Slave] Data byte 102 sent
[I2C Master] Read data byte: 102
[HardDisk] USART Read from Micro address: 1
[HardDisk] USART Write to Micro data: 59
[Microcontroller] new distance: 3186
[Microcontroller] USART Read from USB: 59
[Microcontroller] USART Write to USB: 2
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 14 sent
[I2C Master] Read data byte: 14
[I2C Slave] Data byte 219 sent
[I2C Master] Read data byte: 219
[I2C Slave] Data byte 205 sent
[I2C Master] Read data byte: 205
[HardDisk] USART Read from Micro address: 2
[HardDisk] USART Write to Micro data: 242[Microcontroller] new distance: 3803

[Microcontroller] USART Read from USB: 242
[Microcontroller] USART Write to USB: 3
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 2 sent
[I2C Master] Read data byte: 2
[I2C Slave] Data byte 221 sent
[I2C Master] Read data byte: 221
[I2C Slave] Data byte 219 sent
```

شکل ۳: ادامه اجرای همزمان I2C/C2I و USART برای آدرس های میانی

در شکل ۴، آدرس های بعدی USART نیز ارسال و دریافت شده اند. برای نمونه، آدرس ۳ توسط HardDisk دریافت شده و مقدار ۱۲۶ برای میکروکنترلر ارسال شده است:

```
[HardDisk] USART Read from Micro address: 3
[HardDisk] USART Write to Micro data: 126
[Microcontroller] USART Read from USB: 126
```

این خروجی نشان می دهد که حافظه داخلی HardDisk درست استفاده شده است؛ یعنی آدرس دریافت شده در map داخلی جستجو شده و مقدار متناظر آن به میکروکنترلر برگشته است.

۴.۲۰ تحلیل داده های سنسور در خروجی I2C/C2I

در خروجی مربوط به I2C/C2I، برای هر درخواست، ابتدا آدرس ارسال می شود، سپس سنسور ACK می دهد و بعد سه بایت داده ارسال می کند. به عنوان نمونه، در یکی از تراکنش ها خروجی زیر دیده می شود:

```
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 11 sent
[I2C Master] Read data byte: 11
```

```
[I2C Slave] Data byte 112 sent
[I2C Master] Read data byte: 112
[I2C Slave] Data byte 101 sent
[I2C Master] Read data byte: 101
[Microcontroller] new distance: 2928
```

در این بخش، دو بایت اول داده اصلی سنسور هستند و بایت سوم برای checksum استفاده می شود. میکروکنترلر پس از دریافت سه بایت، مقدار اصلی را بازسازی می کند و اگر checksum معتبر باشد، مقدار نهایی را با پیام new distance چاپ می کند.

```
PS D:\UT\term 8\CPS\CA\CPS\A01\build> . "D:\UT\term 8\CPS\CA\CPS\A01\build\CPS4042.exe"
[I2C Master] Read data byte: 221
[I2C Slave] Data byte 219 sent
[I2C Master] Read data byte: 219
[HardDisk] USART Read from Micro address: 3
[HardDisk] USART Write to Micro data: 126
[Microcontroller] new distance: 733
[Microcontroller] USART Read from USB: 126
[Microcontroller] USART Write to USB: 4
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 11 sent
[I2C Master] Read data byte: 11
[I2C Slave] Data byte 112 sent
[I2C Master] Read data byte: 112
[I2C Slave] Data byte 101 sent
[I2C Master] Read data byte: 101
[HardDisk] USART Read from Micro address: 4
[HardDisk] USART Write to Micro data: 193
[Microcontroller] new distance: 2928
[Microcontroller] USART Read from USB: 193
[Microcontroller] USART Write to USB: 5
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 12 sent
[I2C Master] Read data byte: 12
[I2C Slave] Data byte 47 sent
[I2C Master] Read data byte: 47
[I2C Slave] Data byte 35 sent
[I2C Master] Read data byte: 35
[HardDisk] USART Read from Micro address: [Microcontroller] new distance: 31195
```

شکل ۴: دریافت داده های سنسور و پاسخ های USART برای آدرس های ۴ و ۵

در شکل ۴، علاوه بر چاپ داده های سنسور، پاسخ های USART برای آدرس های ۴ و ۵ نیز دیده می شوند. برای آدرس ۴، مقدار ۱۹۳ و برای آدرس ۵، مقدار ۶۹ از سمت HardDisk به میکروکنترلر برگشته است.

۵.۲۰ تحلیل خروجی های میانی و پیام های همزمان

در برخی بخش های خروجی، بعضی پیام ها روی یک خط یا در کنار هم چاپ شده اند. برای مثال ممکن است پیام مربوط به HardDisk و پیام مربوط به Microcontroller در یک خط دیده شوند. دلیل این اتفاق این است که چند بخش برنامه به صورت همزمان اجرا می شوند و خروجی های std::cout از چند thread مختلف پشت سر هم نوشته می شوند. بنابراین این موضوع به معنی خطای منطقی در پروتکل نیست، بلکه نتیجه اجرای همزمان اجزای شبیه ساز است.


```

PS D:\UT\term 8\CPS\CA\CPS\A01\build> ."D:/UT/term 8/CPS/CA/CPS/A01/build/CPS4042.exe"
[I2C Slave] Data byte 35 sent
[I2C Master] Read data byte: 35
[HardDisk] USART Read from Micro address: [Microcontroller] new distance: 31195
[HardDisk] USART Write to Micro data: 69

[Microcontroller] USART Read from USB: 69
[Microcontroller] USART Write to USB: 6
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 9 sent
[I2C Master] Read data byte: 9
[I2C Slave] Data byte 91 sent
[I2C Master] Read data byte: 91
[I2C Slave] Data byte 82 sent
[I2C Master] Read data byte: 82
[HardDisk] USART Read from Micro address: 6
[HardDisk] USART Write to Micro data: 152
[Microcontroller] new distance: 2395
[Microcontroller] USART Read from USB: 152
[Microcontroller] USART Write to USB: 7
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 9 sent
[I2C Master] Read data byte: 9
[I2C Slave] Data byte 83 sent
[I2C Master] Read data byte: 83
[I2C Slave] Data byte 74 sent
[I2C Master] Read data byte: 74
[HardDisk] USART Read from Micro address: 7
[HardDisk] USART Write to Micro data: [Microcontroller] new distance: 2387214

```

شکل ۵: نمایش همزمانی پیام های I2C و USART در بخش های میانی خروجی

در شکل ۵، آدرس های ۶ و ۷ از طریق USART ارسال شده اند. برای آدرس ۶ مقدار ۱۵۲ و برای آدرس ۷ مقدار ۲۱۴ دریافت شده است. در همین بخش، تراکنش های I2C نیز ادامه دارند و مقادیر سنسور مانند ۲۳۹۵ و ۲۳۸۷ چاپ شده اند. این بخش به خوبی نشان می دهد که هر دو پروتکل در یک اجرای مشترک و همزمان فعال هستند.

۶.۲۰ پایان اجرای USART و دریافت آخرین آدرس ها

در انتهای خروجی، میکروکنترلر آدرس های ۸ و ۹ را برای HardDisk ارسال می کند. برای آدرس ۸، مقدار ۱۸ و برای آدرس ۹، مقدار ۲۵۵ دریافت می شود. این مقادیر مطابق جدول حافظه داخلی HardDisk هستند.

```

PS D:\UT\term 8\CPS\CA\CPS\A01\build> ."D:/UT/term 8/CPS/CA/CPS/A01/build/CPS4042.exe"
[I2C Master] Read data byte: 74
[HardDisk] USART Read from Micro address: 7
[HardDisk] USART Write to Micro data: [Microcontroller] new distance: 2387214

[Microcontroller] USART Read from USB: 214
[Microcontroller] USART Write to USB: 8
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 2 sent
[I2C Master] Read data byte: 2
[I2C Slave] Data byte 112 sent
[I2C Master] Read data byte: 112
[I2C Slave] Data byte 110 sent
[I2C Master] Read data byte: 110
[HardDisk] USART Read from Micro address: 8
[HardDisk] USART Write to Micro data: 18
[Microcontroller] new distance: 624
[Microcontroller] USART Read from USB: 18
[Microcontroller] USART Write to USB: 9
[HardDisk] USART Read from Micro address: 9
[HardDisk] USART Write to Micro data: 255
[Microcontroller] USART Read from USB: 255
PS D:\UT\term 8\CPS\CA\CPS\A01\build>

```

شکل ۶: دریافت پاسخ های نهایی USART برای آدرس های ۸ و ۹

در شکل ۶، پیام های زیر نشان می دهند که آدرس های پایانی نیز با موفقیت پردازش شده اند:

```

[HardDisk] USART Read from Micro address: 8
[HardDisk] USART Write to Micro data: 18
[Microcontroller] USART Read from USB: 18

[HardDisk] USART Read from Micro address: 9
[HardDisk] USART Write to Micro data: 255
[Microcontroller] USART Read from USB: 255

```

بعد از دریافت مقدار مربوط به آدرس ۹، اجرای بخش محدود شده USART تمام می شود، چون در Sketch میکروکنترلر مقدار limit_usart برابر ۱۰ قرار داده شده بود. بنابراین آدرس های ۰ تا ۹ ارسال شده اند و پس از آن آدرس جدیدی ارسال نمی شود.

۷.۲۰ جمع بندی تحلیل خروجی

خروجی اجرا چند نکته اصلی را تایید می کند. اول اینکه بوردها و لینک ها با موفقیت ساخته و متصل شده اند. دوم اینکه پروتکل های I2C/C2I و USART هر دو مقداردی اولیه شده اند. سوم اینکه در ارتباط I2C/C2I، آدرس سنسور درست تشخیص داده شده، ACK ارسال شده و داده سنسور دریافت و اعتبارسنجی شده است. چهارم اینکه در ارتباط USART، میکروکنترلر آدرس های ۰ تا ۹ را ارسال کرده، HardDisk آن ها را خوانده و داده متناظر را برگردانده است.

بنابراین این خروجی نشان می دهد که هر دو بخش اصلی پیاده سازی شده در کنار هم کار می کنند و شبیه ساز توانسته است ارتباط همزمان بین میکروکنترلر، سنسور و هارددیسک فرضی را مدل کند.

در خروجی برنامه، به دلیل اجرای همزمان چند بخش شبیه ساز، گاهی پیام های مربوط به I2C و USART در یک خط یا نزدیک به هم چاپ می شوند. این موضوع به دلیل همزمانی اجرای threadها و استفاده مشترک از cout::std است و به معنی خطای منطقی در ارتباط نیست.

همچنین در بخش USART، ارسال آدرس ها تا مقدار مشخصی محدود شده است. در کد فعلی، میکروکنترلر آدرس های ۰ تا ۹ را ارسال می کند و پس از آن ارسال آدرس جدید انجام نمی شود. این محدودیت فقط برای کنترل تست و خوانا تر شدن خروجی در نظر گرفته شده است.

۲۱ بخش سوم: پیاده سازی و تست C2I/I2C Multiplexer

۱.۲۱ هدف بخش سوم

در بخش سوم پروژه، هدف پیاده سازی یک C2I/I2C Multiplexer بود تا بتوان چند سنسور مبتنی بر پروتکل C2I/I2C را به یک میکروکنترلر متصل کرد. مسئله اصلی در این بخش این است که چند سنسور ممکن است آدرس یکسان داشته باشند. اگر این سنسورها روی یک باس C2I/I2C مشترک قرار بگیرند، هنگام ارسال آدرس، چند سنسور همزمان پاسخ می دهند و روی باس تداخل ایجاد می شود.

برای حل این مشکل، یک MUX بین میکروکنترلر و سنسورها قرار داده شد. میکروکنترلر از طریق USART با MUX ارتباط می گیرد و فقط شماره کانال موردنظر را برای MUX ارسال می کند. سپس MUX همان کانال C2I/I2C را فعال کرده، داده سنسور متصل به آن کانال را می خواند و نتیجه را دوباره از طریق USART به میکروکنترلر ارسال می کند.

ساختار کلی سیستم در این بخش به صورت زیر است:

```
ESP8266 Microcontroller <-> USART <-> I2C/C2I MUX
                                     |
                                     +--> C2I Channel 0 <-> Sensor 0
                                     |
                                     +--> C2I Channel 1 <-> Sensor 1
```

در این پیاده سازی از دو سنسور VL530X استفاده شده است. هر دو سنسور آدرس یکسان 0x29 دارند، اما چون روی دو کانال سخت افزاری جداگانه MUX قرار گرفته اند، تداخلی روی باس ایجاد نمی شود.

۲.۲۱ نکته مهم درباره پروتکل ها

در این بخش، منطق اصلی پروتکل های قبلی تغییر داده نشده است. پروتکل C2I/I2C همان پیاده سازی بخش اول را دارد و پروتکل USART نیز همان پیاده سازی بخش دوم است. تغییرات اصلی فقط در سطح مدل سخت افزاری MUX، نحوه اتصال کانال ها در فایل main و منطق Sketch های مخصوص بخش سوم انجام شده اند.

بنابراین MUX به جای اینکه رفتار پروتکل را تغییر دهد، فقط مشخص می کند که عملیات C2I/I2C روی کدام کانال فیزیکی انجام شود. به بیان دیگر، MUX نقش انتخاب کننده مسیر ارتباطی را دارد.

۳.۲۱ مدل سخت افزاری MUX

MUX برای هر کانال C2I/I2C یک جفت پایه مستقل دارد. یعنی به جای اینکه همه سنسورها به یک SDA/SCL مشترک وصل شوند، هر سنسور روی باس مخصوص خودش قرار می گیرد.

برای کانال صفر، اتصال ها به صورت زیر هستند:

```
I2CMux::scl0 <-> Sensor0::scl
I2CMux::sda0 <-> Sensor0::sda
```

برای کانال یک نیز اتصال ها به شکل زیر هستند:

```
I2CMux::scl1 <-> Sensor1::scl
I2CMux::sda1 <-> Sensor1::sda
```

همچنین خطوط تغذیه و زمین بین میکروکنترلر، MUX و سنسورها مشترک هستند:

```
VDD
GND
```

ارتباط USART نیز بین میکروکنترلر و MUX به صورت ضربدری برقرار می شود:

```
ESP8266::tx <-> I2CMux::rx
ESP8266::rx <-> I2CMux::tx
```

به این ترتیب، میکروکنترلر مستقیماً به سنسورها وصل نیست. ابتدا از طریق USART به MUX فرمان می دهد و سپس MUX ارتباط C2I/I2C با سنسور مناسب را انجام می دهد.

۴.۲۱ کلاس I2CMux

کلاس I2CMux در مسیر زیر پیاده سازی شده است:

```
include/CPS4042/Hardwares/Comm/I2CMux.h
```

این کلاس شامل سه بخش اصلی است:

۱. بخش USART برای ارتباط با میکروکنترلر؛
 ۲. چند نمونه از پروتکل C2I/I2C برای کانال های مختلف؛
 ۳. پایه های سخت افزاری جداگانه برای هر کانال C2I/I2C.
- در این پروژه دو کانال C2I/I2C در نظر گرفته شده است:

```
1 m_i2c0
2 m_i2c1
```

هرکدام از این کانال ها باس منطقی مستقل خود را دارند:

```
1 m_buses[0]
2 m_buses[1]
```

هر سنسور فقط به باس مخصوص کانال خودش وصل می شود. در نتیجه، حتی اگر هر دو سنسور آدرس یکسان داشته باشند، فقط سنسور موجود روی کانال انتخاب شده پاسخ می دهد.

۵.۲۱ منطق Sketch مربوط به MUX

Sketch مربوط به MUX در فایل زیر قرار دارد:

```
include/CPS4042/Sketchs/I2CMultiplexer.h
```

در تابع setup، ابتدا USART راه اندازی می شود. سپس هر دو کانال C2I/I2C با آدرس سنسور VL530X مقداردهی اولیه می شوند:

```
1 node()->usart().init(gpio);
2 node()->i2c(0).init(Sensors::VL530x::address);
3 node()->i2c(1).init(Sensors::VL530x::address);
```

در تابع loop، ابتدا MUX بررسی می کند که آیا از سمت میکروکنترلر فرمانی از طریق USART دریافت شده است یا نه. این فرمان همان شماره کانال است. اگر فرمان دریافت شود، مقدار آن به شماره کانال معتبر تبدیل می شود:

```
1 channel = command % I2CMuxChannelCount;
```

سپس کانال انتخابی فعال می شود:

```
1 node()->selectChannel(m_activeChannel);
```

بعد از انتخاب کانال، MUX از سنسور روی همان کانال سه بایت درخواست می کند:

```
1 node()->i2c(m_activeChannel).requestFrom(Sensors::V1530x::address,
3);
```

این سه بایت شامل دو بایت داده و یک بایت checksum هستند. پس از آماده شدن داده ها، MUX هر بایت را از C2I/I2C می خواند و همان بایت را از طریق USART به میکروکنترلر ارسال می کند:

```
1 data = node()->i2c(m_activeChannel).read();
2 node()->usart().write(data);
```

پس نقش MUX در این بخش، انتخاب کانال، خواندن داده از سنسور همان کانال و forward کردن داده به میکروکنترلر است.

۶.۲۱ منطق Sketch مربوط به میکروکنترلر

Sketch مربوط به میکروکنترلر در فایل زیر قرار دارد:

```
include/CPS4042/Sketchs/MuxMicrocontroller.h
```

میکروکنترلر در هر چند سیکل یک بار، با استفاده از یک شمارنده داخلی، کانال بعدی را انتخاب می کند. این شمارنده بین اجراهای متوالی تابع loop مقدار خود را حفظ می کند و باعث می شود میکروکنترلر به ترتیب از کانال های صفر و یک داده بخواند. روند کار میکروکنترلر در این بخش به صورت زیر است:

۱. انتخاب کانال بعدی؛

۲. ارسال شماره کانال به MUX از طریق USART؛

۳. منتظر ماندن برای دریافت سه بایت پاسخ؛

۴. تشکیل مقدار اصلی سنسور از دو بایت اول؛

۵. بررسی checksum با بایت سوم؛

۶. چاپ مقدار سنسور در صورت معتبر بودن داده.

برای مثال، ابتدا کانال صفر درخواست می شود:

```
[Microcontroller] USART Write to MUX channel request: 0
```

سپس MUX کانال صفر را انتخاب کرده و داده سنسور صفر را می خواند. بعد از آن، میکروکنترلر مقدار دریافتی را چاپ می کند:

```
[Microcontroller] channel 0 sensor value: ...
```

در درخواست بعدی، کانال یک انتخاب می شود:

```
[Microcontroller] USART Write to MUX channel request: 1
```

و داده سنسور یک خوانده می شود:

```
[Microcontroller] channel 1 sensor value: ...
```

۷.۲۱ تولید داده در سنسورها

برای اینکه خروجی هر سنسور قابل تشخیص باشد، سنسورها بازه داده متفاوت دارند:

```
Sensor 0: random values in range 0..20  
Sensor 1: random values in range 50..100
```

به این ترتیب، از روی خروجی برنامه می توان فهمید که MUX واقعا بین کانال ها جابه جا می شود. اگر مقدار کانال صفر همیشه در بازه 0..20 و مقدار کانال یک همیشه در بازه 50..100 باشد، نشان می دهد که انتخاب کانال درست انجام شده است.

۸.۲۱ قالب داده و checksum

هر سنسور مقدار خود را در قالب سه بایت ارسال می کند:

```
Byte 0: first byte of value  
Byte 1: second byte of value  
Byte 2: checksum
```

checksum از اختلاف بایت بزرگ تر و بایت کوچک تر به دست می آید:

```
1 checksum = max(byte0, byte1) - min(byte0, byte1);
```

در سمت میکروکنترلر، پس از دریافت سه بایت، مقدار checksum دوباره محاسبه می شود. اگر checksum دریافتی با checksum محاسبه شده برابر باشد، داده معتبر در نظر گرفته شده و مقدار سنسور چاپ می شود. در غیر این صورت، پیام خطای checksum چاپ می شود. این کار باعث می شود میکروکنترلر مطمئن شود که داده دریافتی از MUX کامل و معتبر است.

۹.۲۱ فایل main بخش سوم

اتصال های سخت افزاری بخش سوم در فایل زیر انجام شده است:

```
src/part3_main.cpp
```

در این فایل، ابتدا نمونه هایی از بوردها ساخته می شود:

```
1 Boards::Esp8266 esp8266;  
2 Comm::I2CMux mux;  
3 Sensors::Vl530x sensor0;  
4 Sensors::Vl530x sensor1;
```

سپس اتصال های لازم برای تغذیه، زمین، USART و دو کانال C2I/I2C ساخته می شوند. در این بخش، برای خطوط مشترکی که بیش از دو قطعه به آن ها متصل می شوند، از Bus استفاده شده است. برای مثال، VDD و GND بین میکروکنترلر، مالتی پلکسر و دو سنسور مشترک هستند، بنابراین با `std::make_shared<Bus>()` ساخته شده اند. همچنین برای هر کانال C2I/I2C نیز از Bus مستقل استفاده شده است تا خط های SCL و SDA آن کانال بتوانند بین MUX و سنسور همان کانال مشترک باشند. در مقابل، برای ارتباط های دو نقطه ای USART از Link استفاده شده است.

```

1 auto linkVdd = std::make_shared<Bus>();
2 auto linkGnd = std::make_shared<Bus>();
3
4 auto linkTxEspRxMux = std::make_shared<Link>();
5 auto linkRxEspTxMux = std::make_shared<Link>();
6
7 auto linkScl0 = std::make_shared<Bus>();
8 auto linkSda0 = std::make_shared<Bus>();
9
10 auto linkScl1 = std::make_shared<Bus>();
11 auto linkSda1 = std::make_shared<Bus>();

```

در نتیجه، کانال صفر و کانال یک نه تنها از نظر منطقی، بلکه از نظر اتصال فیزیکی شبیه ساز نیز از هم جدا هستند. هر سنسور فقط به SCL/SDA کانال خودش متصل می شود و به باس کانال دیگر دسترسی ندارد. سنسور صفر فقط به کانال صفر وصل می شود:

```

1 mux.gpio().scl0.attachLink(linkScl0);
2 mux.gpio().sda0.attachLink(linkSda0);
3
4 sensor0.gpio().scl.attachLink(linkScl0);
5 sensor0.gpio().sda.attachLink(linkSda0);

```

سنسور یک نیز فقط به کانال یک وصل می شود:

```

1 mux.gpio().scl1.attachLink(linkScl1);
2 mux.gpio().sda1.attachLink(linkSda1);
3
4 sensor1.gpio().scl.attachLink(linkScl1);
5 sensor1.gpio().sda.attachLink(linkSda1);

```

در انتها Sketch های مربوط به میکروکنترلر، MUX و سنسورها ساخته و اجرا می شوند:

```

1 micro.start();
2 i2cMux.start();
3 range0.start();
4 range1.start();
5
6 Application::exec();

```

با فراخوانی Application::exec()، محیط شبیه سازی فعال می ماند و اجزای سیستم به صورت پیوسته کار می کنند. برنامه تا زمانی که کاربر با Ctrl+C آن را متوقف نکند، به خواندن دوره ای سنسورها ادامه می دهد.

۱۰.۲۱ جزئیات پیاده سازی بخش سوم

در این بخش، فایل های اصلی پروژه که رفتار ارتباطی سیستم را شکل می دهند، با جزئیات بیشتری بررسی می شوند. هدف این بخش این است که مشخص شود هر فایل چه نقشی در معماری کلی پروژه دارد، چه کلاس ها و توابعی در آن تعریف شده اند و هر بخش چگونه در اجرای نهایی سه سناریوی C2I/I2C، USART و I2C MUX استفاده می شود.

۱۰.۱۰.۲۱ فایل I2CMux.h: مدل سخت افزاری مالتی پلکسر

فایل I2CMux.h مدل سخت افزاری برد I2C MUX را تعریف می کند. این برد در بخش سوم پروژه بین میکروکنترلر و دو سنسور قرار می گیرد. از یک طرف با میکروکنترلر از طریق USART ارتباط دارد و از طرف

دیگر دو کانال مستقل C2I/I2C برای اتصال به سنسورها فراهم می کند. در ابتدای فایل، فضای نام Comm تعریف شده و مقدار ولتاژ کاری مورد برابر 3.3V در نظر گرفته شده است:

```
1 namespace Comm
2 {
3     using I2CMuxVoltage = VoltageLevel3_3v;
4     inline constexpr std::uint8_t I2CMuxChannelCount = 2;
```

ثابت I2CMuxChannelCount تعداد کانال های MUX را مشخص می کند. در پیاده سازی فعلی، این مقدار برابر ۲ است؛ یعنی MUX دو کانال مستقل برای ارتباط C2I/I2C دارد.

کلاس SelectedSdaPin یکی از نکات مهم در طراحی I2CMux این است که پروتکل I2C Master قبلی همچنان انتظار دارد با یک پایه SDA و یک پایه SCL کار کند. اما در بخش سوم، ما دو جفت پایه فیزیکی داریم:

```
sda0 / scl0
sda1 / scl1
```

برای اینکه بدون تغییر دادن منطق اصلی پروتکل I2C Master بتوانیم بین این کانال ها جابه جا شویم، کلاس SelectedSdaPin تعریف شده است. این کلاس نقش یک پایه SDA مجازی را دارد. یعنی از بیرون، پروتکل فکر می کند فقط روی یک SDA می نویسد، اما این کلاس بسته به کانال انتخاب شده، داده را به sda0 یا sda1 منتقل می کند. تابع bind آدرس پایه های واقعی کانال صفر و یک را ذخیره می کند:

```
1 void bind(ChannelPin* channel0, ChannelPin* channel1)
2 {
3     m_channels[0] = channel0;
4     m_channels[1] = channel1;
5 }
```

تابع select شماره کانال فعال را تعیین می کند:

```
1 void select(std::uint8_t channel)
2 {
3     m_selectedChannel = channel % I2CMuxChannelCount;
4 }
```

استفاده از عملگر % باعث می شود اگر مقدار فرمان از تعداد کانال ها بزرگ تر باشد، همچنان به یک کانال معتبر تبدیل شود. برای مثال اگر دو کانال داشته باشیم، مقدار ۲ دوباره به کانال ۰ و مقدار ۳ به کانال ۱ تبدیل می شود.

تابع activePin پایه واقعی مربوط به کانال فعال را برمی گرداند:

```
1 ChannelPin* activePin() const
2 {
3     return m_channels[m_selectedChannel % I2CMuxChannelCount];
4 }
```

در نهایت، توابع write(Bit) و write(Byte) داده را روی پایه فعال می نویسند:

```
1 void write(Bit bit)
2 {
3     if(auto* pin = activePin()) pin->write(bit);
4 }
5
```



```

6 void write(Byte byte)
7 {
8     if(auto* pin = activePin()) pin->write(byte);
9 }

```

بنابراین SelectedSdaPin یک لایه واسط است که عملیات نوشتن روی SDA را به کانال فیزیکی درست منتقل می کند.

کلاس SelectedSclPin مشابه SelectedSdaPin است، با این تفاوت که برای خط کلاک SCL استفاده می شود. در I2C، خط SCL برای زمان بندی ارتباط استفاده می شود و تغییر لبه های آن باید به سنسور کانال فعال برسد. در این کلاس نیز ابتدا پایه های فیزیکی دو کانال با تابع bind ثبت می شوند:

```

1 void bind(ChannelPin* channel0, ChannelPin* channel1)
2 {
3     m_channels[0] = channel0;
4     m_channels[1] = channel1;
5 }

```

سپس با تابع select کانال فعال مشخص می شود. تفاوت اصلی این کلاس با SelectedSdaPin این است که برای SCL تابع nextEdge تعریف شده است:

```

1 void nextEdge(Bit bit)
2 {
3     if(auto* pin = activePin()) pin->nextEdge(bit);
4 }

```

یعنی هر زمان کلاک ارتباطی پردازنده تغییر کند، لبه جدید فقط روی SCL کانال فعال منتشر می شود. به این ترتیب سنسورهای کانال های دیگر کلاک دریافت نمی کنند و وارد تراکنش نمی شوند.

ساختار I2CMuxGpio ساختار I2CMux همه پایه های بورد I2CMux را تعریف می کند. این پایه ها به سه گروه اصلی تقسیم می شوند. گروه اول پایه های تغذیه و زمین هستند:

```

1 Pins::Vdd<WorkingVoltageTp> vdd {BR, BTR, "I2CMux::vdd"};
2 Pins::Gnd<WorkingVoltageTp> gnd {BR, BTR, "I2CMux::gnd"};

```

گروه دوم پایه های USART برای ارتباط با میکروکنترلر هستند:

```

1 Pins::Rx<WorkingVoltageTp> rx {BR, BTR, "I2CMux::rx"};
2 Pins::Tx<WorkingVoltageTp> tx {BR, BTR, "I2CMux::tx"};

```

در اتصال فیزیکی، TX میکروکنترلر به RX مالتی پلکسر و TX مالتی پلکسر به RX میکروکنترلر وصل می شود. گروه سوم پایه های مربوط به C2I/I2C هستند. ابتدا دو پایه انتخابی یا مجازی داریم:

```

1 SelectedSdaPin<WorkingVoltageTp> sda {BR, BTR, "I2CMux::selected::sda"};
2 SelectedSclPin<WorkingVoltageTp> scl {BR, BTR, "I2CMux::selected::scl"};

```

این دو پایه برای سازگاری با پروتکل I2C Master استفاده می شوند. سپس پایه های واقعی دو کانال تعریف می شوند:

```

1 Pins::Sda<WorkingVoltageTp> sda0 {BR, BTR, "I2CMux::ch0::sda"};
2 Pins::Scl<WorkingVoltageTp> scl0 {BR, BTR, "I2CMux::ch0::scl"};
3
4 Pins::Sda<WorkingVoltageTp> sda1 {BR, BTR, "I2CMux::ch1::sda"};
5 Pins::Scl<WorkingVoltageTp> scl1 {BR, BTR, "I2CMux::ch1::scl"};

```

بنابراین MUX در ظاهر یک بورد واحد است، اما از سمت سنسورها دو باس مستقل دارد.

کلاس اصلی I2CMux کلاس I2CMux از کلاس عمومی Board ارث بری می کند:

```

1 class I2CMux
2 : public Board<BaudRates::NotSpecified,
3               BitRates::same(BaudRates::NotSpecified),
4               Frequency::F320khz,
5               I2CMuxVoltage,
6               I2CMuxGpio>

```

در این تعریف، نرخ BaudRate مشخص نشده است و BitRate نیز از همان مقدار ساخته می شود. فرکانس بورد برابر F320khz است و ولتاژ کاری آن 3.3V در نظر گرفته شده است. در سازنده کلاس، ابتدا پایه های انتخابی sda و scl به پایه های واقعی کانال صفر و یک متصل می شوند:

```

1 m_gpio.sda.bind(&m_gpio.sda0, &m_gpio.sda1);
2 m_gpio.scl.bind(&m_gpio.scl0, &m_gpio.scl1);
3 selectChannel(0);

```

بنابراین در شروع برنامه، کانال پیش فرض برابر صفر است. سپس کلاک ارتباطی پردازنده به SCL انتخابی وصل می شود:

```

1 m_processor->communicationClockChanged.connect(
2   [this](Bit edge) { m_gpio.scl.nextEdge(edge); });

```

این خط بسیار مهم است. چون هر بار که کلاک ارتباطی پردازنده تغییر کند، لبه جدید از طریق SelectedSclPin فقط به کانال فعال منتقل می شود. در نتیجه، اگر کانال صفر انتخاب شده باشد، کلاک روی scl0 می رود و اگر کانال یک انتخاب شده باشد، کلاک روی scl1 می رود. بعد از آن، برای هر کانال یک باس مستقل I2C تنظیم می شود:

```

1 m_i2c0.setBus(&m_buses[0]);
2 m_i2c1.setBus(&m_buses[1]);

```

این بخش باعث می شود ارتباط منطقی کانال صفر و کانال یک از هم جدا باشد. یعنی حتی اگر دو سنسور آدرس یکسان داشته باشند، هر کدام روی باس مربوط به خودشان پاسخ می دهند. در نهایت، سه پروتکل روی پردازنده MUX نصب می شوند:

```

1 m_processor->installProtocol(&m_usart);
2 m_processor->installProtocol(&m_i2c0);
3 m_processor->installProtocol(&m_i2c1);

```

پس I2CMux همزمان سه نقش ارتباطی دارد: یک ارتباط USART با میکروکنترلر، یک I2C Master برای کانال صفر و یک I2C Master برای کانال یک.

توابع دسترسی در I2CMux تابع usart دسترسی به پروتکل USART را فراهم می کند:

```

1 Protocols::Usart<I2CMux, Gpio>& usart() const
2 {
3     return m_usart;
4 }

```

تابع i2c با گرفتن شماره کانال، پروتکل I2C Master همان کانال را برمی گرداند:

```

1 Protocols::I2CMaster<I2CMux, Gpio>& i2c(std::uint8_t channel)
   const
2 {
3     channel %= I2CMuxChannelCount;
4     return (channel == 0) ? m_i2c0 : m_i2c1;
5 }

```

تابع bus نیز برای گرفتن آدرس باس منطقی کانال استفاده می شود:

```

1 Protocols::I2CBusLine* bus(std::uint8_t channel) const
2 {
3     channel %= I2CMuxChannelCount;
4     return const_cast<Protocols::I2CBusLine*>(&m_buses[channel]);
5 }

```

این تابع در part3_main.cpp استفاده می شود تا سنسور هر کانال به باس منطقی همان کانال وصل شود. تابع selectChannel نیز کانال فعال SDA و SCL را تغییر می دهد:

```

1 void selectChannel(std::uint8_t channel) const
2 {
3     channel %= I2CMuxChannelCount;
4     auto& gpio = const_cast<I2CMux*>(this)->m_gpio;
5     gpio.sda.select(channel);
6     gpio.scl.select(channel);
7 }

```

بنابراین انتخاب کانال فقط یک متغیر ساده نیست، بلکه مسیر فیزیکی نوشتن روی SDA و تولید لبه روی SCL را تغییر می دهد.

۲.۱۰.۲۱ فایل I2CMultiplexer.h: رفتار کاربردی مالتی پلکسر

فایل I2CMultiplexer.h رفتار اجرایی برد I2CMux را در قالب یک Sketch تعریف می کند. خود I2CMux.h سخت افزار و پروتکل های قابل استفاده را معرفی می کند، اما اینکه MUX در هر سیکل چه کاری انجام دهد، در این Sketch مشخص می شود. کلاس اصلی این فایل به صورت زیر تعریف شده است:

```

1 class I2CMultiplexer : public AbstractSketch<Comm::I2CMux>

```

یعنی این Sketch روی یک نود از نوع Comm::I2CMux اجرا می شود.

تابع setup در I2CMultiplexer در تابع setup، ابتدا پروتکل USART مقداردهی اولیه می شود:

```

1 node()->usart().init(gpio);

```

سپس هر دو کانال I2C با آدرس سنسور VL530X مقداردهی می شوند:

```

1 node()->i2c(0).init(Sensors::VL530x::address);
2 node()->i2c(1).init(Sensors::VL530x::address);

```

از آنجا که هر دو سنسور از نوع VL530X هستند، آدرس هر دو برابر 0x29 است. اما چون هر کانال باس مستقل خودش را دارد، استفاده از آدرس یکسان مشکلی ایجاد نمی کند.

دریافت فرمان کانال از میکروکنترلر در تابع loop، اگر MUX منتظر پاسخ سنسور نباشد و داده ای از سمت USART آماده باشد، آن داده به عنوان فرمان انتخاب کانال خوانده می شود:

```

1 if(!m_waitingForSensor && node()->usart().isDataAvailable())
2 {
3     Byte command = node()->usart().read();
4     std::uint8_t channel =
5         static_cast<UByte>(command) % Comm::I2CMuxChannelCount;

```

شرط `m_waitingForSensor` باعث می شود وقتی MUX هنوز در حال دریافت داده از سنسور قبلی است، فرمان جدید را شروع نکند. بنابراین تراکنش ها با هم قاطی نمی شوند. بعد از خواندن فرمان، شماره کانال فعال ذخیره می شود و شمارنده بایت های دریافتی صفر می شود:

```

1 m_activeChannel = channel;
2 m_receivedBytes = 0;
3 m_waitingForSensor = true;

```

سپس کانال فیزیکی متناظر انتخاب می شود و درخواست خواندن سه بایت از سنسور روی همان کانال ارسال می شود:

```

1 node()->selectChannel(m_activeChannel);
2 node()->i2c(m_activeChannel).requestFrom(Sensors::VL530x::address,
3);

```

به این ترتیب، MUX پس از دریافت یک بایت فرمان از میکروکنترلر، یک تراکنش I2C کامل را روی کانال انتخاب شده شروع می کند.

خواندن داده سنسور و ارسال آن به میکروکنترلر بعد از شروع تراکنش I2C، MUX منتظر آماده شدن داده از سمت سنسور می ماند. اگر داده در کانال فعال آماده باشد، یک بایت خوانده می شود:

```

1 Byte data = node()->i2c(m_activeChannel).read();

```

سپس همان بایت بدون تغییر از طریق USART برای میکروکنترلر ارسال می شود:

```

1 node()->usart().write(data);
2 m_receivedBytes++;

```

بنابراین I2CMultiplexer خودش مقدار سنسور را تفسیر نمی کند و checksum را بررسی نمی کند. نقش آن فقط این است که داده را از سنسور کانال فعال بخواند و همان داده را به میکروکنترلر منتقل کند. وقتی سه بایت کامل منتقل شد، وضعیت انتظار پایان می یابد:

```

1 if(m_receivedBytes >= 3)
2 {
3     m_receivedBytes = 0;
4     m_waitingForSensor = false;
5 }

```

به این ترتیب، MUX آماده دریافت فرمان کانال بعدی از میکروکنترلر می شود.

متغیرهای داخلی I2CMultiplexer در انتهای کلاس سه متغیر داخلی وجود دارد:

```
1 std::uint8_t m_activeChannel {0};
2 std::uint8_t m_receivedBytes {0};
3 bool m_waitingForSensor {false};
```

متغیر m_activeChannel کانالی را نگه می‌دارد که در حال حاضر انتخاب شده است. متغیر m_receivedBytes تعداد بایت‌هایی را نگه می‌دارد که از سنسور دریافت و برای میکروکنترلر ارسال شده‌اند. متغیر m_waitingForSensor مشخص می‌کند که آیا MUX در میانه یک تراکنش سنسور قرار دارد یا آماده دریافت فرمان جدید است.

۱۱.۲۱ فایل MuxMicrocontroller.h: رفتار میکروکنترلر در تست I2C MUX

فایل MuxMicrocontroller.h رفتار میکروکنترلر را در سناریوی بخش سوم تعریف می‌کند. در این سناریو، میکروکنترلر مستقیماً با سنسورها از طریق I2C ارتباط نمی‌گیرد، بلکه فقط از طریق USART با MUX صحبت می‌کند. کلاس اصلی این فایل به صورت زیر تعریف شده است:

```
1 class MuxMicroController : public AbstractSketch<Boards::Esp8266>
```

یعنی این Sketch روی بورد ESP8266 اجرا می‌شود.

تابع setup در MuxMicroController در تابع setup فقط پروتکل USART میکروکنترلر مقداردهی می‌شود:

```
1 node()->usart().init(gpio);
```

در این Sketch برخلاف میکروکنترلر بخش اول، I2C میکروکنترلر مقداردهی نمی‌شود؛ چون مسیر I2C توسط خود MUX مدیریت می‌شود.

ارسال دوره‌ای شماره کانال در تابع loop، شمارنده سیکل در هر بار اجرا افزایش پیدا می‌کند:

```
1 m_cycleCount++;
```

اگر میکروکنترلر منتظر پاسخ قبلی نباشد و تعداد سیکل‌ها به مضرب ۳۰۰ رسیده باشد، یک درخواست جدید ارسال می‌شود:

```
1 if(!m_waitingForResponse && m_cycleCount % 300 == 0)
2 {
3     m_activeChannel = m_nextChannel;
4     m_nextChannel = (m_nextChannel + 1) % Comm::I2CMuxChannelCount
5     ;
```

در این بخش، m_activeChannel کانالی است که برای درخواست فعلی استفاده می‌شود و m_nextChannel برای درخواست بعدی آماده می‌شود. استفاده از Comm::I2CMuxChannelCount باعث می‌شود انتخاب کانال به صورت چرخشی بین صفر و یک انجام شود.

قبل از ارسال درخواست، شمارنده بایت‌های دریافتی و ByteStream پاک می‌شوند:

```
1 m_bytesReceived = 0;
2 m_stream.clear();
3 m_waitingForResponse = true;
```

سپس شماره کانال فعال از طریق USART برای MUX ارسال می‌شود:

```
1 node()->usart().write(static_cast<Byte>(m_activeChannel));
```

بنابراین میکروکنترلر فقط یک بایت فرمان می‌فرستد و منتظر می‌ماند که MUX سه بایت پاسخ را برگرداند.

دریافت سه بایت پاسخ از MUX در ادامه تابع loop، اگر داده ای از طریق USART آماده باشد، خوانده می شود:

```
1 if(node()->usart().isDataAvailable())
2 {
3     Byte data = node()->usart().read();
4     m_bytesReceived++;
```

اگر بایت دریافتی، بایت اول یا دوم باشد، داخل ByteStream قرار می گیرد:

```
1 if(m_bytesReceived <= 2)
2 {
3     m_stream << data;
4 }
```

دو بایت اول مقدار اصلی سنسور را تشکیل می دهند. وقتی بایت سوم دریافت شود، به عنوان checksum در نظر گرفته می شود:

```
1 else if(m_bytesReceived == 3)
2 {
3     Byte receivedChecksum = data;
```

سپس اگر ByteStream آماده باشد، مقدار ۱۶ بیتی سنسور بازسازی می شود:

```
1 std::uint16_t value = m_stream.take();
```

بعد از بازسازی مقدار، دو بایت اصلی دوباره استخراج می شوند:

```
1 Byte byte0 = getByte<0>(value);
2 Byte byte1 = getByte<1>(value);
```

سپس checksum مورد انتظار از اختلاف بایت بزرگ تر و کوچک تر محاسبه می شود:

```
1 Byte expectedChecksum =
2     static_cast<Byte>(std::max(ub0, ub1) - std::min(ub0, ub1));
```

اگر checksum دریافتی با مقدار محاسبه شده برابر باشد، مقدار سنسور چاپ می شود:

```
1 if(receivedChecksum == expectedChecksum)
2 {
3     std::cout << "[Microcontroller] channel "
4               << static_cast<int>(m_activeChannel)
5               << " sensor value: " << value << std::endl;
6 }
```

در غیر این صورت، پیام خطای checksum چاپ می شود:

```
1 else
2 {
3     std::cout << "[Microcontroller] checksum mismatch on channel "
4               << static_cast<int>(m_activeChannel) << std::endl;
5 }
```

در پایان دریافت سه بایت، شمارنده صفر می شود و میکروکنترلر از حالت انتظار خارج می شود:

```
1 m_bytesReceived = 0;
2 m_waitingForResponse = false;
```

بنابراین این Sketch یک چرخه کامل درخواست و پاسخ را مدیریت می کند: ارسال کانال، دریافت سه بایت، بازسازی مقدار، بررسی checksum و چاپ نتیجه.

۱۰۱۱.۲۱ فایل part3_main.cpp: اجرای مستقل سناریوی I2C MUX

فایل part3_main.cpp سناریوی کامل بخش سوم را اجرا می کند. در این سناریو، یک میکروکنترلر، یک I2C MUX و دو سنسور VL530X وجود دارند. هر دو سنسور آدرس یکسان دارند، اما روی دو کانال جداگانه قرار گرفته اند.

در ابتدا چهار قطعه اصلی ساخته می شوند:

```
1 Boards::Esp8266 esp8266;  
2 Comm::I2CMux mux;  
3 Sensors::VL530x sensor0;  
4 Sensors::VL530x sensor1;
```

سپس برای تغذیه و زمین از Bus استفاده می شود:

```
1 auto linkVdd = std::make_shared<Bus>();  
2 auto linkGnd = std::make_shared<Bus>();
```

دلیل استفاده از Bus این است که تغذیه و زمین بین چند قطعه مشترک هستند؛ یعنی هم میکروکنترلر، هم MUX و هم دو سنسور به آن ها وصل می شوند.

برای ارتباط USART بین میکروکنترلر و MUX از دو Link استفاده می شود:

```
1 auto linkTxEspRxMux = std::make_shared<Link>();  
2 auto linkRxEspTxMux = std::make_shared<Link>();
```

این دو لینک ارتباط دوطرفه سریال را به صورت ضربدری برقرار می کنند.
برای کانال های I2C نیز چهار Bus مستقل ساخته می شود:

```
1 auto linkScl0 = std::make_shared<Bus>();  
2 auto linkSda0 = std::make_shared<Bus>();  
3  
4 auto linkScl1 = std::make_shared<Bus>();  
5 auto linkSda1 = std::make_shared<Bus>();
```

در اینجا، linkScl0/linkSda0 باس فیزیکی کانال صفر و linkScl1/linkSda1 باس فیزیکی کانال یک را تشکیل می دهند.

اتصال USART بین ESP8266 و MUX به صورت زیر انجام می شود:

```
1 esp8266.gpio().tx.attachLink(linkTxEspRxMux);  
2 esp8266.gpio().rx.attachLink(linkRxEspTxMux);  
3  
4 mux.gpio().rx.attachLink(linkTxEspRxMux);  
5 mux.gpio().tx.attachLink(linkRxEspTxMux);
```

بنابراین داده ای که میکروکنترلر روی TX خود می نویسد، از RX مالتی پلکسر خوانده می شود و داده ای که مالتی پلکسر روی TX خود می نویسد، از RX میکروکنترلر خوانده می شود.
سپس تغذیه و زمین برای دو سنسور نیز مشترک می شود:

```
1 sensor0.gpio().vdd.attachLink(linkVdd);  
2 sensor0.gpio().gnd.attachLink(linkGnd);  
3  
4 sensor1.gpio().vdd.attachLink(linkVdd);  
5 sensor1.gpio().gnd.attachLink(linkGnd);
```

بعد از آن، سنسور صفر به کانال صفر MUX وصل می شود:

```

1 mux.gpio().scl0.attachLink(linkScl0);
2 mux.gpio().sda0.attachLink(linkSda0);
3 sensor0.gpio().scl.attachLink(linkScl0);
4 sensor0.gpio().sda.attachLink(linkSda0);

```

و سنسور یک به کانال یک متصل می شود:

```

1 mux.gpio().scl1.attachLink(linkScl1);
2 mux.gpio().sda1.attachLink(linkSda1);
3 sensor1.gpio().scl.attachLink(linkScl1);
4 sensor1.gpio().sda.attachLink(linkSda1);

```

این بخش مهم ترین قسمت اتصال سخت افزاری بخش سوم است. چون نشان می دهد دو سنسور به یک SDA/SCL مشترک وصل نشده اند، بلکه هرکدام باس مستقل خود را دارند. در پایان، Sketch های مربوط به میکروکنترلر و مالتی پلکسر ساخته می شوند:

```

1 MuxMicroController micro(&esp8266);
2 I2CMultiplexer i2cMux(&mux);

```

برای دو سنسور نیز از RangeSensor استفاده شده است:

```

1 RangeSensor range0(&sensor0, mux.bus(0), 0, 20, 101, "Sensor-0");
2 RangeSensor range1(&sensor1, mux.bus(1), 50, 100, 202, "Sensor-1");
;

```

در اینجا، سنسور صفر به باس منطقی کانال صفر وصل می شود و مقدارهایی در بازه 0..20 تولید می کند. سنسور یک به باس منطقی کانال یک وصل می شود و مقدارهایی در بازه 50..100 تولید می کند. مقدارهای 101 و 202 نیز برای کنترل تولید داده و متفاوت بودن رفتار دو سنسور استفاده شده اند. نام های Sensor-0 و Sensor-1 هم در خروجی برنامه کمک می کنند که تشخیص دهیم کدام سنسور راه اندازی شده است. در نهایت، هر چهار Sketch شروع می شوند:

```

1 micro.start();
2 i2cMux.start();
3 range0.start();
4 range1.start();

```

و با اجرای Application::exec() برنامه فعال باقی می ماند:

```

1 return Application::exec();

```

بنابراین part3_main.cpp کل سناریوی بخش سوم را می سازد: اتصال میکروکنترلر به MUX با USART، اتصال MUX به دو سنسور با دو باس مستقل I2C، و اجرای رفتار انتخاب کانال و خواندن داده از سنسورهای هم آدرس.

۱۲.۲۱ خروجی کامل تست مستقل بخش سوم

برای بررسی مستقل بخش سوم، یک اجرای جداگانه فقط برای سناریوی I2C MUX انجام شد. در این تست، هدف این بود که مطمئن شویم میکروکنترلر می تواند از طریق USART شماره کانال را برای MUX ارسال کند، MUX کانال درست را انتخاب کند، داده سنسور همان کانال را از طریق C2I/I2C بخواند و سپس سه بایت پاسخ را از طریق USART به میکروکنترلر برگرداند.

خروجی کامل این تست در فایل I2CMUX_out.txt ذخیره شده است. برای اینکه گزارش کامل و قابل بررسی باشد، خروجی کامل تست با دستور زیر داخل گزارش آورده می شود. کافی است فایل I2CMUX_out.txt کنار فایل اصلی HW.tex قرار داشته باشد:


```

Esp8266::Processor created.
one instance of Esp8266 created.
I2CMux::processor created.
one instance of I2CMux created.
Vl530x::processor created.
one instance of Vl530x created.
Vl530x::processor created.
one instance of Vl530x created.
linkVdd: header Esp8266::vdd1 attached to link successfully.
Esp8266::vdd1: linkVdd attached successfully
linkGnd: header Esp8266::gnd1 attached to link successfully.
Esp8266::gnd1: linkGnd attached successfully
linkTxEspRxMux: header Esp8266::tx attached to link successfully.
Esp8266::tx: linkTxEspRxMux attached successfully
linkRxEspTxMux: header Esp8266::rx attached to link successfully.
Esp8266::rx: linkRxEspTxMux attached successfully
linkVdd: header I2CMux::vdd attached to link successfully.
I2CMux::vdd: linkVdd attached successfully
linkGnd: header I2CMux::gnd attached to link successfully.
I2CMux::gnd: linkGnd attached successfully
linkTxEspRxMux: header I2CMux::rx attached to link successfully.
I2CMux::rx: linkTxEspRxMux attached successfully
linkRxEspTxMux: header I2CMux::tx attached to link successfully.
I2CMux::tx: linkRxEspTxMux attached successfully
linkVdd: header Vl530x::vdd attached to link successfully.
Vl530x::vdd: linkVdd attached successfully
linkGnd: header Vl530x::gnd attached to link successfully.
Vl530x::gnd: linkGnd attached successfully
linkVdd: header Vl530x::vdd attached to link successfully.
Vl530x::vdd: linkVdd attached successfully
linkGnd: header Vl530x::gnd attached to link successfully.
Vl530x::gnd: linkGnd attached successfully
linkScl0: header I2CMux::ch0::scl attached to link successfully.
I2CMux::ch0::scl: linkScl0 attached successfully
linkSda0: header I2CMux::ch0::sda attached to link successfully.
I2CMux::ch0::sda: linkSda0 attached successfully
linkScl0: header Vl530x::scl attached to link successfully.
Vl530x::scl: linkScl0 attached successfully
linkSda0: header Vl530x::sda attached to link successfully.
Vl530x::sda: linkSda0 attached successfully
linkScl1: header I2CMux::ch1::scl attached to link successfully.
I2CMux::ch1::scl: linkScl1 attached successfully
linkSda1: header I2CMux::ch1::sda attached to link successfully.
I2CMux::ch1::sda: linkSda1 attached successfully
linkScl1: header Vl530x::scl attached to link successfully.
Vl530x::scl: linkScl1 attached successfully
linkSda1: header Vl530x::sda attached to link successfully.
Vl530x::sda: linkSda1 attached successfully
[USART] Initialized
MuxMicroController setup completed.
[USART] Initialized

```

```

[I2C Master] Initialized I2C target 0x29
[I2C Master] Initialized I2C target 0x29
I2C MUX setup completed.
[Microcontroller] USART Write to MUX channel request: 0
[I2C Slave] Initialized I2C own address 0x29
[Sensor-0] setup completed, range 0..20
[I2C MUX] USART Read from Microcontroller channel command: 0 ->
    selected channel 0
[I2C MUX] selected I2C channel 0
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 0 sent
[I2C Master] Read data byte: 0
[I2C Slave] Data byte 10 sent
[I2C Master] Read data byte: 10
[I2C Slave] Data byte 10 sent
[I2C Master] Read data byte: 10
[I2C MUX] USART Write to Microcontroller: 0 from I2C channel 0
[I2C MUX] USART Write to Microcontroller: 10 from I2C channel 0
[I2C MUX] USART Write to Microcontroller: 10 from I2C channel 0
[Microcontroller] USART Read from MUX: 0 (byte 1/3 for channel 0)
[Microcontroller] USART Read from MUX: 10 (byte 2/3 for channel 0)
[Microcontroller] USART Read from MUX: 10 (byte 3/3 for channel 0)
[Microcontroller] channel 0 sensor value: 10
[Microcontroller] USART Write to MUX channel request: 1
[I2C MUX] USART Read from Microcontroller channel command: 1 ->
    selected channel 1
[I2C MUX] selected I2C channel 1
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Initialized I2C own address 0x29
[Sensor-1] setup completed, range 50..100
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 0 sent
[I2C Master] Read data byte: 0
[I2C Slave] Data byte 61 sent
[I2C Master] Read data byte: 61
[I2C Slave] Data byte 61 sent
[I2C Master] Read data byte: 61
[I2C MUX] USART Write to Microcontroller: 0 from I2C channel 1
[I2C MUX] USART Write to Microcontroller: 61 from I2C channel 1
[I2C MUX] USART Write to Microcontroller: 61 from I2C channel 1
[Microcontroller] USART Read from MUX: 0 (byte 1/3 for channel 1)
[Microcontroller] USART Read from MUX: 61 (byte 2/3 for channel 1)
[Microcontroller] USART Read from MUX: 61 (byte 3/3 for channel 1)
[Microcontroller] channel 1 sensor value: 61
[Microcontroller] USART Write to MUX channel request: 0
[I2C MUX] USART Read from Microcontroller channel command: 0 ->

```

```

selected channel 0
[I2C MUX] selected I2C channel 0
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
[I2C Slave] Data byte 0 sent
[I2C Master] Read data byte: 0
[I2C Slave] Data byte 11 sent
[I2C Master] Read data byte: 11
[I2C Slave] Data byte 11 sent
[I2C Master] Read data byte: 11
[I2C MUX] USART Write to Microcontroller: 0 from I2C channel 0
[I2C MUX] USART Write to Microcontroller: 11 from I2C channel 0
[I2C MUX] USART Write to Microcontroller: 11 from I2C channel 0
[Microcontroller] USART Read from MUX: 0 (byte 1/3 for channel 0)
[Microcontroller] USART Read from MUX: 11 (byte 2/3 for channel 0)
[Microcontroller] USART Read from MUX: 11 (byte 3/3 for channel 0)
[Microcontroller] channel 0 sensor value: 11

```

این خروجی نشان می دهد که ابتدا چهار قطعه اصلی سیستم ساخته شده اند: یک میکروکنترلر ESP8266، یک برد I2CMux و دو سنسور VL530X. سپس لینک های تغذیه، زمین، USART و دو کانال مستقل C2I/I2C متصل شده اند. وجود پیام های مربوط به linkScl0، linkSda0، linkScl1 و linkSda1 نشان می دهد که سنسور صفر و سنسور یک روی دو باس فیزیکی جداگانه قرار گرفته اند و روی یک باس مشترک نیستند.

در ادامه، میکروکنترلر ابتدا کانال صفر را درخواست می کند. MUX فرمان کانال صفر را از طریق USART دریافت کرده، کانال صفر را انتخاب می کند و سپس از طریق C2I/I2C با سنسور همان کانال ارتباط می گیرد. در این تراکنش، سنسور سه بایت 0، 10 و 10 ارسال می کند. دو بایت اول مقدار سنسور را تشکیل می دهند و بایت سوم checksum است. چون اختلاف دو بایت اول برابر 10 است، checksum معتبر است و میکروکنترلر مقدار نهایی کانال صفر را برابر 10 چاپ می کند.

در تراکنش بعدی، میکروکنترلر کانال یک را درخواست می کند. این بار MUX کانال یک را انتخاب می کند و با سنسور دوم ارتباط می گیرد. در این مرحله نیز آدرس سنسور همان 0x29 است، اما چون سنسور دوم روی باس جداگانه کانال یک قرار دارد، تداخلی با سنسور اول ایجاد نمی شود. سنسور کانال یک سه بایت 0، 61 و 61 ارسال می کند و میکروکنترلر مقدار نهایی کانال یک را برابر 61 گزارش می کند.

در ادامه، میکروکنترلر دوباره کانال صفر را درخواست می کند. MUX دوباره به کانال صفر برمی گردد و مقدار جدید سنسور صفر را می خواند. در این تراکنش، مقدار نهایی کانال صفر برابر 11 چاپ شده است. این رفتار نشان می دهد که MUX فقط یک بار کانال را انتخاب نمی کند، بلکه می تواند در طول اجرای برنامه بین کانال ها جابه جا شود.

از روی این خروجی می توان نتیجه گرفت که تست مستقل بخش سوم موفق بوده است. چون:

۱. میکروکنترلر و MUX از طریق USART درست به هم وصل شده اند.
۲. MUX فرمان انتخاب کانال را از میکروکنترلر دریافت می کند.
۳. کانال های صفر و یک C2I/I2C به صورت فیزیکی جدا هستند.
۴. هر دو سنسور با وجود داشتن آدرس یکسان 0x29 بدون تداخل پاسخ می دهند.
۵. مقدار کانال صفر در بازه 0..20 و مقدار کانال یک در بازه 50..100 قرار دارد.
۶. داده سنسور از مسیر C2I/I2C به MUX و سپس از مسیر USART به میکروکنترلر منتقل می شود.

۷. checksum داده های دریافتی معتبر است و مقدار نهایی به درستی چاپ می شود.

بنابراین این تست مستقل نشان می دهد که بخش سوم پروژه، یعنی پیاده سازی C2I/I2C Multiplexer، از نظر انتخاب کانال، جداسازی باس ها، خواندن داده سنسورهای هم آدرس و ارسال پاسخ به میکروکنترلر درست کار می کند.

در فایل خروجی، در بعضی پیام ها به جای خط تیره، عبارت ΓÇê دیده می شود. این موضوع مربوط به encoding ترمینال یا فایل خروجی است و به معنی خطا در منطق پروتکل نیست.

۱۳.۲۱ بررسی خروجی اجرای بخش سوم

پس از اجرای بخش سوم، ابتدا بوردهای موردنیاز ساخته می شوند. در خروجی مشاهده می شود که یک نمونه از Esp8266، یک نمونه از I2CMux و دو نمونه از سنسور V1530x ایجاد شده اند:

```
Esp8266::Processor created.  
one instance of Esp8266 created.  
I2CMux::processor created.  
one instance of I2CMux created.  
V1530x::processor created.  
one instance of V1530x created.  
V1530x::processor created.  
one instance of V1530x created.
```

سیس اتصال های تغذیه، زمین و USART بین میکروکنترلر و MUX برقرار می شوند. در خروجی، اتصال TX میکروکنترلر به RX در MUX و اتصال RX میکروکنترلر به TX در MUX دیده می شود:

```
linkTxEspRxMux: header Esp8266::tx attached to link successfully.  
linkTxEspRxMux: header I2CMux::rx attached to link successfully.  
linkRxEspTxMux: header Esp8266::rx attached to link successfully.  
linkRxEspTxMux: header I2CMux::tx attached to link successfully.
```

مهم ترین بخش خروجی مربوط به کانال های C2I/I2C مستقل است. در این پیاده سازی، سنسور صفر به کانال صفر MUX و سنسور یک به کانال یک MUX وصل شده است. بنابراین برخلاف اتصال باس مشترک، برای هر کانال لینک های جداگانه SCL و SDA وجود دارد:

```
linkSc10: header I2CMux::ch0::scl attached to link successfully.  
linkSda0: header I2CMux::ch0::sda attached to link successfully.  
linkSc10: header V1530x::scl attached to link successfully.  
linkSda0: header V1530x::sda attached to link successfully.  
  
linkSc11: header I2CMux::ch1::scl attached to link successfully.  
linkSda1: header I2CMux::ch1::sda attached to link successfully.  
linkSc11: header V1530x::scl attached to link successfully.  
linkSda1: header V1530x::sda attached to link successfully.
```

این بخش نشان می دهد که دو سنسور با آدرس یکسان روی یک باس مشترک قرار نگرفته اند، بلکه هرکدام از طریق یک کانال سخت افزاری جداگانه به MUX متصل شده اند.

۱۴.۲۱ تحلیل تراکنش کانال صفر

پس از راه اندازی USART و I2C، میکروکنترلر ابتدا کانال صفر را درخواست می کند:

```
[Microcontroller] USART Write to MUX channel request: 0
[I2C MUX] USART Read from Microcontroller channel command: 0 ->
    selected channel 0
[I2C MUX] selected I2C channel 0
```

سپس MUX روی کانال صفر، آدرس سنسور 0x29 را روی باس C2I/I2C ارسال می کند. سنسور متصل به همان کانال آدرس را تشخیص داده و ACK می دهد:

```
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
```

بعد از دریافت ACK، سه بایت داده از سنسور خوانده می شود:

```
[I2C Slave] Data byte 0 sent
[I2C Master] Read data byte: 0
[I2C Slave] Data byte 10 sent
[I2C Master] Read data byte: 10
[I2C Slave] Data byte 10 sent
[I2C Master] Read data byte: 10
```

در این نمونه، مقدار سنسور برابر ۱۰ است. سه بایت ارسال شده به ترتیب عبارت اند از:

0, 10, 10

دو بایت اول مقدار سنسور را تشکیل می دهند و بایت سوم checksum است. چون اختلاف بایت بزرگ تر و کوچک تر برابر ۱۰ است، checksum معتبر است. سپس MUX همین سه بایت را از طریق USART برای میکروکنترلر ارسال می کند:

```
[I2C MUX] USART Write to Microcontroller: 0 from I2C channel 0
[I2C MUX] USART Write to Microcontroller: 10 from I2C channel 0
[I2C MUX] USART Write to Microcontroller: 10 from I2C channel 0
```

میکروکنترلر سه بایت را از USART می خواند و پس از بررسی checksum، مقدار نهایی را چاپ می کند:

```
[Microcontroller] USART Read from MUX: 0 (byte 1/3 for channel 0)
[Micocontroller] USART Read from MUX: 10 (byte 2/3 for channel 0)
[Micocontroller] USART Read from MUX: 10 (byte 3/3 for channel 0)
[Micocontroller] channel 0 sensor value: 10
```

۱۵.۲۱ تحلیل تراکنش کانال یک

در مرحله بعد، میکروکنترلر کانال یک را درخواست می کند:

```
[Microcontroller] USART Write to MUX channel request: 1
[I2C MUX] USART Read from Microcontroller channel command: 1 ->
    selected channel 1
[I2C MUX] selected I2C channel 1
```

MUX این بار ارتباط C2I/I2C را روی کانال یک انجام می دهد. سنسور دوم نیز با همان آدرس 0x29 پاسخ می دهد، اما چون روی باس جداگانه قرار دارد، تداخلی با سنسور اول ایجاد نمی شود:

```
[I2C Master] Address byte 0x53 (READ) sent on SDA
[I2C Slave] Address 0x29 R received
[I2C Slave] ACK sent
[I2C Master] ACK received
```

سپس سه بایت مربوط به سنسور کانال یک خوانده می شود:

```
[I2C Slave] Data byte 0 sent
[I2C Master] Read data byte: 0
[I2C Slave] Data byte 61 sent
[I2C Master] Read data byte: 61
[I2C Slave] Data byte 61 sent
[I2C Master] Read data byte: 61
```

در اینجا مقدار سنسور برابر ۶۱ است. سه بایت 0, 61, 61 از MUX به میکروکنترلر منتقل می شوند:

```
[I2C MUX] USART Write to Microcontroller: 0 from I2C channel 1
[I2C MUX] USART Write to Microcontroller: 61 from I2C channel 1
[I2C MUX] USART Write to Microcontroller: 61 from I2C channel 1
```

و سپس میکروکنترلر همان سه بایت را دریافت کرده و مقدار نهایی را چاپ می کند:

```
[Microcontroller] USART Read from MUX: 0 (byte 1/3 for channel 1)
[Microcontroller] USART Read from MUX: 61 (byte 2/3 for channel 1)
[Microcontroller] USART Read from MUX: 61 (byte 3/3 for channel 1)
[Microcontroller] channel 1 sensor value: 61
```

۱۶.۲۱ تکرار انتخاب کانال صفر

در ادامه، برنامه دوباره به کانال صفر برمی گردد و مقدار بعدی را می خواند:

```
[Microcontroller] USART Write to MUX channel request: 0
[I2C MUX] USART Read from Microcontroller channel command: 0 ->
    selected channel 0
[I2C MUX] selected I2C channel 0
...
[Microcontroller] channel 0 sensor value: 11
```

این خروجی نشان می دهد که میکروکنترلر به صورت دوره ای کانال های MUX را انتخاب می کند و در هر بار فقط داده سنسور همان کانال دریافت می شود.

۱۷.۲۱ خلاصه نتایج خروجی بخش سوم

خلاصه چند تراکنش اصلی تست در جدول زیر آمده است:

جدول ۳: خلاصه تراکنش های بخش سوم

Requested Channel	Sensor Range	Returned Bytes	Final Value
0	0..20	0, 10, 10	10
1	50..100	0, 61, 61	61
0	0..20	0, 11, 11	11

این جدول نشان می دهد که در هر بار انتخاب کانال، داده از سنسور همان کانال خوانده شده است. همچنین checksum در هر سه نمونه معتبر بوده است.

۱۸.۲۱ نتیجه گیری از خروجی

از روی خروجی می توان چند نکته را تایید کرد:

۱. ارتباط USART بین میکروکنترلر و MUX درست برقرار شده است.
۲. MUX فرمان انتخاب کانال را از میکروکنترلر دریافت می کند.
۳. MUX بعد از دریافت فرمان، کانال C2I/I2C متناظر را انتخاب می کند.
۴. هر دو سنسور آدرس یکسان 0x29 دارند، اما چون روی کانال های جداگانه قرار گرفته اند، تداخل ایجاد نمی شود.
۵. داده سنسور از طریق C2I/I2C به MUX رسیده و سپس از طریق USART به میکروکنترلر ارسال می شود.
۶. مقدارهای کانال صفر در بازه 0..20 و مقدارهای کانال یک در بازه 50..100 هستند؛ بنابراین داده ها قابل تفکیک اند و انتخاب کانال به درستی انجام شده است.
۷. برنامه به صورت پیوسته اجرا می شود و میکروکنترلر به ترتیب از کانال های صفر و یک داده می خواند.

۲۲ پاسخ سوالات بخش اول: پروتکل آدرس محور C2I/I2C

در این بخش، سوالات مربوط به پیاده سازی پروتکل آدرس محور C2I/I2C پاسخ داده می شوند. این سوالات بیشتر به ماهیت باس بودن پروتکل، محدودیت های نسخه ساده شده و ابزارهای کمکی انتقال داده چندبایتی مربوط هستند.

۱.۲۲ سوال اول: مشکل اتصال چند سنسور با آدرس مشابه روی یک باس

در پروتکل های باس محور مانند C2I/I2C، چند قطعه می توانند از خطوط مشترک SDA و SCL استفاده کنند، اما شرط مهم این است که هر قطعه آدرس قابل تشخیص خودش را داشته باشد. اگر دو یا چند سنسور با آدرس یکسان روی یک باس قرار بگیرند، میکروکنترلر دیگر نمی تواند تشخیص دهد که کدام سنسور باید به درخواست پاسخ دهد.

در چنین حالتی، وقتی میکروکنترلر آن آدرس را روی باس می فرستد، همه سنسورهایی که آن آدرس را دارند خودشان را مخاطب پیام فرض می کنند. بنابراین ممکن است چند سنسور همزمان ACK بدهند و بعد همزمان روی خط SDA داده بنویسند. اگر داده های سنسورها یکسان نباشند، مقدار خوانده شده توسط میکروکنترلر نامعتبر یا غیرقابل پیش بینی می شود. حتی اگر در بعضی لحظه ها مقدار یکسانی نوشته شود، باز هم میکروکنترلر نمی داند داده مربوط به کدام سنسور بوده است.

پس مشکل اصلی در اتصال سنسورهای هم آدرس، **تداخل آدرس و تداخل داده روی باس** است. راه حل معمول این مشکل در سیستم های واقعی، استفاده از آدرس های متفاوت، پایه های انتخاب آدرس، چند باس جداگانه، یا استفاده از I2C Multiplexer است تا در هر لحظه فقط یک مسیر یا یک سنسور هم آدرس فعال باشد.

۲.۲۲ سوال دوم: چهار اشکال نسخه ساده شده نسبت به C2I/I2C واقعی

نسخه پیاده سازی شده در این تمرین، یک مدل ساده شده از پروتکل آدرس محور است و هدف آن بیشتر فهم منطق کلی ارتباط، آدرس دهی، دریافت ACK و انتقال داده است. این نسخه نسبت به یک پروتکل واقعی مانند I2C چند محدودیت مهم دارد:

۱. نبود شرط های واقعی شروع و پایان: در I2C واقعی، شروع و پایان هر تراکنش با شرایط مشخصی روی خطوط SDA و SCL مشخص می شود که به آن ها Start Condition و Stop Condition گفته می شود. در نسخه ساده شده تمرین، تراکنش بیشتر با وضعیت های داخلی برنامه مدیریت می شود و این شرط های دقیق فیزیکی پیاده سازی نشده اند.

۲. نبود ACK/NACK برای هر بایت: در I2C واقعی، بعد از هر بایت معمولا یک بیت ACK یا NACK ارسال می شود تا مشخص شود گیرنده بایت را دریافت کرده است یا نه. در نسخه ساده شده، ACK بیشتر برای تایید آدرس استفاده شده و کنترل دقیق بعد از هر بایت مانند نسخه واقعی وجود ندارد.

۳. نبود داوری و مدیریت کامل تداخل روی باس: در پروتکل واقعی، مخصوصا وقتی چند master وجود داشته باشد، سازوکارهایی مثل arbitration و تشخیص مشغول بودن باس لازم است. در نسخه ساده شده تمرین، فرض اصلی این است که فقط یک master وجود دارد و مدیریت کامل رقابت روی باس پیاده سازی نشده است.

۴. ساده شدن مدل الکتریکی خط داده: در I2C واقعی، خطوط معمولا به صورت open-drain/open-collector و با مقاومت pull-up کار می کنند. در شبیه ساز ما، این جزئیات الکتریکی به صورت کامل مدل نشده و بیشتر منطق نرم افزاری خواندن و نوشتن روی لینک پیاده سازی شده است.

۵. نبود امکاناتی مانند clock stretching و زمان بندی دقیق: در I2C واقعی، slave می تواند در بعضی شرایط با نگه داشتن خط SCL ارتباط را کند کند. همچنین محدودیت های دقیق زمانی مانند setup time و hold time وجود دارند. در نسخه ساده شده، این موارد به صورت دقیق پیاده سازی نشده اند.

بنابراین نسخه تمرین برای نمایش منطق اصلی ارتباط مناسب است، اما همه جزئیات سخت افزاری، زمانی و کنترلی I2C واقعی را پوشش نمی دهد.

۳.۲۲ سوال سوم: کارکرد ByteStream، ByteVector و getByte

در بخش اول تمرین، مقدار سنسور عددی بین ۰ تا ۴۰۰۰ است. این بازه در یک بایت جا نمی شود، چون یک بایت فقط می تواند مقدارهای ۰ تا ۲۵۵ را نگهداری کند. بنابراین مقدار سنسور باید به چند بایت تقسیم شود و در سمت گیرنده دوباره به مقدار اصلی تبدیل شود. برای ساده تر شدن این کار، از ابزارهایی مانند ByteStream، ByteVector و تابع getByte استفاده می شود.

۱. تابع getByte: این تابع برای جدا کردن یک بایت مشخص از یک مقدار چندبایتی استفاده می شود. مثلا اگر مقدار سنسور از نوع uint16_t باشد، می توان با این تابع بایت اول و بایت دوم آن را جدا کرد و جداگانه از طریق پروتکل فرستاد.

۲. کلاس ByteVector: این کلاس یا ساختار کمکی برای نگهداری مجموعه ای از بایت های حاصل از یک مقدار چندبایتی استفاده می شود. به کمک آن می توان مقدار اصلی را به چند بایت قابل ارسال تبدیل کرد. در سمت سنسور، چنین ابزاری کمک می کند مقدار ۱۶ بیتی به دو بایت تقسیم شود.

۳. کلاس ByteStream: این کلاس در سمت گیرنده برای جمع کردن چند بایت و بازسازی مقدار اصلی استفاده می شود. مثلا میکروکنترلر دو بایت داده سنسور را یکی یکی دریافت می کند و داخل ByteStream قرار می دهد. وقتی تعداد بایت های لازم کامل شد، ByteStream می تواند مقدار اصلی را بازسازی کند.

به طور خلاصه، getByte و ByteVector بیشتر در مرحله شکستن مقدار چندبایتی به بایت های قابل ارسال کاربرد دارند، در حالی که ByteStream بیشتر در مرحله جمع کردن بایت های دریافتی و بازسازی مقدار اصلی استفاده می شود. این ابزارها باعث می شوند Sketch ها لازم نباشد درگیر همه جزئیات شیفت دادن بیت ها و ترکیب دستی بایت ها شوند.

۲۳ پاسخ سوالات بخش دوم: پروتکل USART Full Duplex

در این بخش، سوالات مربوط به پروتکل USART پاسخ داده می شوند. این سوالات به ساختار فریم، نبود خط کلاک و امکان پیاده سازی نرم افزاری USART با استفاده از پایه های دیجیتال مربوط هستند.

۱.۲۳ سوال اول: چرا بیت های شروع و پایان با بیت های داده اشتباه گرفته نمی شوند؟

در USART، گیرنده بیت ها را بدون دانستن ساختار فریم تفسیر نمی کند، بلکه یک ماشین حالت مشخص برای دریافت دارد. خط ارتباطی در حالت بیکار معمولاً روی مقدار ۱ قرار دارد. وقتی فرستنده می خواهد ارسال را شروع کند، خط را به ۰ می برد. این تغییر از حالت بیکار به صفر، به عنوان Start Bit شناخته می شود و گیرنده با دیدن آن متوجه می شود که یک فریم جدید شروع شده است.

بعد از تشخیص Start Bit، گیرنده دیگر هر صفر یا یک بعدی را به عنوان شروع یا پایان جدید تفسیر نمی کند. بلکه دقیقاً طبق ساختار فریم، تعداد مشخصی بیت را به عنوان بیت های داده می خواند. در این تمرین، تعداد بیت های داده ۸ بیت در نظر گرفته شده است. بعد از دریافت این ۸ بیت، گیرنده انتظار دارد بیت بعدی Stop Bit باشد و مقدار آن ۱ باشد.

بنابراین دلیل اشتباه گرفته نشدن بیت های شروع و پایان با بیت های داده این است که موقعیت هر بیت در فریم مشخص است. Start Bit فقط در حالت بیکار و در ابتدای فریم معنا دارد. بیت های داده فقط بعد از Start Bit و به تعداد مشخص خوانده می شوند. Stop Bit نیز فقط بعد از پایان بیت های داده بررسی می شود. پس اگر داخل داده مقدار صفر یا یک وجود داشته باشد، با Start Bit یا Stop Bit اشتباه گرفته نمی شود، چون گیرنده در آن لحظه در حالت دریافت داده است، نه در حالت انتظار برای شروع یا پایان فریم.

۲.۲۳ سوال دوم: با نبود خط کلاک، USART چگونه داده ها را تشخیص می دهد؟

USART در حالت آسنکرون خط کلاک جداگانه ندارد. در عوض، دو طرف ارتباط باید از قبل روی یک نرخ ارسال مشترک یا همان Baud Rate توافق داشته باشند. یعنی فرستنده و گیرنده هر دو می دانند هر بیت تقریباً چه مدت زمانی روی خط باقی می ماند.

گیرنده ابتدا منتظر Start Bit می ماند. وقتی افت خط از ۱ به ۰ را تشخیص داد، زمان بندی داخلی خودش را فعال می کند. سپس با استفاده از همان Baud Rate، در زمان های مناسب از خط نمونه برداری می کند. معمولاً نمونه برداری در میانه بازه زمانی هر بیت انجام می شود تا احتمال خطا کمتر شود. به این ترتیب، گیرنده می تواند بیت های داده را یکی یکی تشخیص دهد، حتی بدون اینکه خط کلاک جداگانه ای بین دو دستگاه وجود داشته باشد.

در واقع Start Bit نقش همزمان سازی اولیه را دارد. بعد از آن، گیرنده با ساعت داخلی خودش و با فرض ثابت بودن Baud Rate بیت های بعدی را می خواند. به همین دلیل، اگر نرخ ارسال دو طرف خیلی با هم تفاوت داشته باشد یا زمان بندی دچار خطای زیاد شود، گیرنده ممکن است بیت ها را اشتباه بخواند. در شبیه ساز این تمرین، این موضوع ساده تر مدل شده است و فرض می شود زمان بندی دو طرف برای ارسال و دریافت فریم هماهنگ است.

۳.۲۳ سوال سوم: آیا پیاده سازی نرم افزاری USART با پین های دیجیتال ممکن است؟

بله، پیاده سازی نرم افزاری USART با استفاده از پین های دیجیتال ممکن است. به این روش معمولاً bit-banging یا Software UART گفته می شود. در این روش، برنامه به جای استفاده از سخت افزار اختصاصی USART/UART، خودش مقدار پین خروجی را در زمان های مناسب تغییر می دهد و از پین ورودی نیز در زمان های مناسب نمونه برداری می کند.

اما این کار نسبت به پیاده سازی نرم افزاری پروتکل های سنکرون مانند C2I/I2C سخت تر است. دلیل اصلی این است که USART خط کلاک ندارد و گیرنده باید بیت ها را با زمان بندی دقیق بخواند. بنابراین برنامه باید بتواند با دقت زمانی بالا Start Bit را تشخیص دهد، سپس بیت های داده را در فاصله های زمانی درست نمونه برداری کند و در پایان Stop Bit را بررسی کند. در سرعت های پایین، پیاده سازی نرم افزاری USART معمولاً امکان پذیر است، اما محدودیت هایی دارد:

۱. نیاز به زمان بندی دقیق: اگر اجرای برنامه، وقفه ها یا کارهای دیگر باعث تاخیر شوند، نمونه برداری از بیت ها در زمان اشتباه انجام می شود.

۲. مصرف پردازنده بیشتر: چون پردازنده باید خودش بیت ها را با زمان بندی دقیق کنترل کند، درگیر ارتباط سریال می شود.

۳. سخت تر بودن Full Duplex: ارسال و دریافت همزمان به صورت نرم افزاری سخت تر از ارتباط یک طرفه یا نیمه دوطرفه است، چون برنامه باید همزمان هم بیت ارسال کند و هم بیت دریافت کند.

۴. محدودیت در نرخ ارسال: در Baud Rate های بالا، پیاده سازی نرم افزاری ممکن است پایدار نباشد و بهتر است از سخت افزار اختصاصی استفاده شود.

در مقابل، در پروتکل هایی مانند C2I/I2C، مخصوصاً وقتی میکروکنترلر نقش master دارد، پیاده سازی نرم افزاری ساده تر است، چون master می تواند خط SCL را خودش کنترل کند و هر وقت لازم بود ارتباط را کندتر جلو ببرد. اما در USART، چون کلاک جداگانه وجود ندارد، زمان بندی باید دقیق تر رعایت شود. بنابراین پاسخ نهایی این است که پیاده سازی نرم افزاری USART ممکن است، ولی برای سرعت های بالا، ارتباط پایدار، دریافت و ارسال همزمان و کاهش بار پردازنده، استفاده از سخت افزار UART/USART انتخاب مطمئن تر و حرفه ای تری است.

۲۴ پاسخ سوالات بخش سوم

۱.۲۴ سوال اول: تابع reverse در Byte.h چه کاری انجام می دهد و آیا در بوردهای واقعی هم چنین عملیاتی استفاده می شود؟

تابع reverse ترتیب بیت های یک بایت را برعکس می کند. یعنی بیت کم ارزش تر را به جای بیت پر ارزش تر قرار می دهد و همین کار را برای بقیه بیت ها نیز انجام می دهد. برای مثال اگر مقدار باینری زیر را داشته باشیم:

00010010

بعد از reverse شدن، نتیجه به شکل زیر خواهد بود:

01001000

این تابع زمانی کاربرد دارد که ترتیب ارسال بیت ها با ترتیب مورد انتظار گیرنده متفاوت باشد. برای مثال، یک پروتکل ممکن است داده را از بیت کم ارزش تر ارسال کند، در حالی که بخش دیگری از سیستم انتظار داشته باشد داده از بیت پر ارزش تر خوانده شود.

در بوردهای واقعی نیز مسئله ترتیب بیت ها وجود دارد، اما معمولا این کار مستقیما با چنین تابعی در سطح نرم افزار انجام نمی شود. در سخت افزارهای واقعی، واحدهای ارتباطی مانند UART، SPI یا I2C معمولا دارای shift register هستند و ترتیب ارسال یا دریافت بیت ها توسط سخت افزار یا تنظیمات peripheral کنترل می شود.

در بعضی پروتکل ها نیز ترتیب بیت ها از قبل در استاندارد مشخص است. برای مثال، در I2C معمولا بیت ها از سمت MSB ارسال می شوند، در حالی که در UART/USART معمولا LSB زودتر ارسال می شود. بنابراین تابع reverse در شبیه ساز برای حل اختلاف ترتیب بیت ها مفید است، اما در دنیای واقعی معمولا این مسئله با تنظیمات سخت افزاری peripheral، shift register یا driver سطح پایین مدیریت می شود.

۲.۲۴ سوال دوم: توابع takeNthBit در Bit.h چه کاری انجام می دهند و تفاوت آن ها چیست؟

تابع takeNthBit برای استخراج بیت شماره n از یک مقدار عددی استفاده می شود. این تابع یک mask می سازد، آن را به اندازه n بیت shift می دهد و سپس با عملگر & بررسی می کند که بیت موردنظر صفر است یا یک. ایده ساده این تابع به صورت زیر است:

```
1 Bit takeNthBit(Byte value, int n)
2 {
3     return (value >> n) & 1;
4 }
```

نسخه اول تابع به شکل run-time کار می کند:

```
1 takeNthBit(value, n)
```

در این نسخه، مقدار و شماره بیت هنگام اجرای برنامه داده می شوند. اگر شماره بیت از عرض نوع داده بیشتر باشد، تابع پیام خطا چاپ می کند و مقدار Bit::X برمی گرداند. این نسخه برای زمانی مناسب است که شماره بیت در زمان اجرا مشخص می شود. نسخه دوم تابع به شکل compile-time نوشته شده است:

```
1 takeNthBit<value, n>()
```

در این نسخه، مقدار و شماره بیت به صورت template parameter داده می شوند. بنابراین بررسی معتبر بودن شماره بیت در زمان کامپایل انجام می شود. اگر شماره بیت معتبر نباشد، برنامه در زمان کامپایل خطا می دهد. این نسخه برای مقادیری مناسب است که از قبل ثابت هستند و می خواهیم خطاها قبل از اجرای برنامه تشخیص داده شوند.

پس تفاوت اصلی این دو نسخه در زمان بررسی و نوع استفاده است:

- نسخه run-time: مناسب برای مقادارها و اندیس هایی که هنگام اجرای برنامه مشخص می شوند.
 - نسخه compile-time: مناسب برای مقادارها و اندیس های ثابت که در زمان کامپایل قابل بررسی هستند.
- در پروژه ما، این تابع برای پیاده سازی پروتکل هایی مثل USART کاربرد دارد؛ زیرا در USART باید یک بایت به بیت های جداگانه تبدیل شود و هر بیت در یک مرحله روی خط TX نوشته شود.

۳.۲۴ سوال سوم: تابع attachPinToCommunicationClock از کلاس Board چه کاری انجام می دهد؟

تابع attachPinToCommunicationClock یکی از پایه های دیجیتال برد را به کلاک ارتباطی پردازنده متصل می کند. این تابع به صورت template نوشته شده و شماره پایه را به عنوان template parameter دریافت می کند. ابتدا بررسی می شود که برد از نوع Drived نباشد؛ یعنی خودش بتواند کلاک تولید کند. سپس بررسی می شود که شماره پایه معتبر باشد و پایه انتخاب شده از نوع Pins::Digital باشد. بعد از آن، پایه موردنظر به سیگنال communicationClockChanged پردازنده وصل می شود:

```
1 m_processor->communicationClockChanged.connect(  
2     [this, &pin](Bit edge) { pin.write(edge); }  
3 );
```

بنابراین هر بار که کلاک ارتباطی پردازنده تغییر کند، مقدار جدید کلاک روی آن پایه نوشته می شود. کاربرد این تابع در پروتکل هایی است که یک خط کلاک نیاز دارند؛ مانند C2I/I2C که در آن خط SCL برای همگام سازی فرستنده و گیرنده استفاده می شود. در واقع این تابع باعث می شود یک پایه دیجیتال برد نقش خروجی کلاک ارتباطی را بازی کند و تغییرات کلاک پردازنده روی آن پایه و در نتیجه روی لینک متصل به آن منتشر شود. این موضوع در بخش C2I/I2C و مخصوصا بخش I2C MUX اهمیت دارد. چون در I2C، خط SCL نقش کلاک مشترک را دارد و slave ها با همین کلاک متوجه می شوند چه زمانی باید داده را بخوانند یا پاسخ دهند. اگر پایه کلاک درست به communication clock وصل نشود، ممکن است از نظر فیزیکی پایه ها به هم وصل باشند، اما ارتباط از نظر زمانی درست جلو نرود. تفاوت این مسئله با USART این است که USART در حالت آسنکرون خط کلاک جداگانه ندارد. در USART دو طرف با Baud Rate مشترک هماهنگ می شوند، اما در C2I/I2C خط SCL خودش نقش کلاک مشترک را بازی می کند.

۲۵ تصمیم های فنی مهم

۱.۲۵ استفاده از ماشین حالت

برای هر دو پروتکل از ماشین حالت استفاده شد. این تصمیم باعث شد رفتار پروتکل ها مرحله به مرحله و قابل کنترل باشد. در پروتکل های ارتباطی، معمولا همه مراحل در یک لحظه انجام نمی شوند. مثلا در I2C ابتدا آدرس ارسال می شود، سپس ACK دریافت می شود و بعد داده خوانده می شود. در USART نیز ابتدا start bit، بعد بیت های داده و در پایان stop bit ارسال می شود. ماشین حالت برای مدل کردن چنین رفتارهایی مناسب است.

۲.۲۵ جدا کردن لایه کاربردی از لایه پروتکل

در طراحی انجام شده، Sketch ها درگیر جزئیات بیت ها و مرحله های داخلی پروتکل نمی شوند. Sketch فقط توابع سطح بالایی مثل read، write و requestFrom را صدا می زند. جزئیات ارسال آدرس، ACK، فریم USART و مدیریت صف ها در خود پروتکل انجام می شود. این جداسازی باعث شد کد تمیزتر و قابل نگهداری تر شود.

۳.۲۵ استفاده از صف و بافر

در هر دو پروتکل، داده ها با صف و بافر مدیریت شدند. این کار باعث شد تولید داده و مصرف داده وابسته به اجرای همزمان در یک سیکل نباشد. برای مثال، سنسور می تواند داده را در صف قرار دهد و پروتکل در زمان مناسب آن را ارسال کند. در سمت گیرنده نیز داده دریافت شده در بافر می ماند تا Sketch آن را بخواند.

۴.۲۵ اعتبارسنجی داده سنسور

برای داده سنسور از checksum استفاده شد. این تصمیم باعث شد میکروکنترلر بتواند داده نامعتبر را تشخیص دهد. هرچند این checksum ساده است، اما برای تمرین و بررسی صحت انتقال چندبایتی مناسب بود.

۲۶ چالش های پیاده سازی

۱.۲۶ هماهنگی بین Master و Slave

در ارتباط I2C/C2I، master و slave باید وضعیت مشترک ارتباط را به شکل هماهنگ ببینند. از آنجا که هر بورد می تواند در thread جدا اجرا شود، احتمال ناهماهنگی وجود داشت. برای کاهش این مشکل، وضعیت مشترک bus با mutex محافظت شد.

۲.۲۶ ترتیب بایت ها در داده ۱۶ بیتی

مقدار سنسور ۱۶ بیتی بود و باید به دو بایت تقسیم می شد. یکی از چالش ها این بود که ترتیب ارسال و بازسازی بایت ها درست رعایت شود. استفاده از ByteStream و checksum به ما کمک کرد این بخش را کنترل کنیم.

۳.۲۶ اتصال درست TX و RX

در USART اتصال پایه ها باید به صورت ضربدری انجام شود. یعنی TX یک طرف به RX طرف دیگر وصل شود. اگر این اتصال اشتباه باشد، ارتباط به درستی شکل نمی گیرد. این نکته در تابع main رعایت شد.

۴.۲۶ اجرای همزمان دو پروتکل روی میکروکنترلر

میکروکنترلر در این پروژه همزمان با دو ارتباط کار می کند: ارتباط با سنسور و ارتباط با هارددیسک فرضی. برای اینکه این دو منطق با هم قاطی نشوند، در Sketch میکروکنترلر دو تابع جداگانه برای مدیریت آن ها نوشته شد.

۲۷ فایل های استفاده شده در پروژه

۱.۲۷ فایل های آماده و زیرساختی

فایل هایی مانند Object.h، Board.h، Processor.h، Transmitter.h، Pin.h، Link.h، Queue.h، ByteStream.h، Wave.h و فایل های مربوط به Units شامل Bit.h، Byte.h، Voltage.h، Frequency.h و BaudRate.h از قبل در اختیار ما قرار گرفته بودند. این فایل ها زیرساخت اصلی شبیه ساز را می ساختند و ما از آن ها برای پیاده سازی بخش های خواسته شده استفاده کردیم. در این گزارش، این فایل ها فقط در حد نقش کلی آن ها در معماری پروژه معرفی شده اند.

۲.۲۷ فایل های اصلی پیاده سازی پروژه

فایل های اصلی پروژه در چند گروه قرار می گیرند. گروه اول مربوط به پروتکل ها است، گروه دوم بوردها و قطعات سخت افزاری را تعریف می کند، گروه سوم Sketch های اجرایی هر قطعه را مشخص می کند و گروه چهارم فایل های main هستند که سناریوهای مختلف تست را می سازند.

- I2C.h: پیاده سازی منطق پروتکل C2I/I2C شامل سمت master و slave، ارسال آدرس، دریافت ACK، خواندن و نوشتن داده و مدیریت باس مشترک.

- Usart.h: پیاده سازی پروتکل USART با فریم شامل start bit، هشت بیت داده و stop bit، همراه با ماشین حالت های مستقل برای ارسال و دریافت.
- Esp8266.h: تعریف برد ESP8266 به عنوان میکروکنترلر اصلی پروژه، همراه با پایه های GPIO، ارتباط C2I/I2C و ارتباط USART.
- VL530X.h: تعریف سنسور VL530X با پایه های تغذیه، زمین، SCL و SDA و آدرس ثابت 0x29.
- Usb.h: تعریف برد Usb برای تست ارتباط USART و استفاده در سناریوی HardDisk.
- I2CMux.h: تعریف برد I2C MUX، شامل پایه های USART، دو کانال مستقل C2I/I2C، پایه های انتخابی SDA/SCL و دو باس منطقی مستقل برای کانال صفر و یک.
- Microcontroller.h: تعریف رفتار میکروکنترلر در سناریوی ترکیبی، شامل خواندن داده از سنسور از طریق C2I/I2C و ارسال آدرس به HardDisk از طریق USART.
- UsartMicroController.h: تعریف رفتار میکروکنترلر برای تست مستقل USART، شامل ارسال آدرس های متوالی و دریافت پاسخ از HardDisk.
- MuxMicrocontroller.h: تعریف رفتار میکروکنترلر در سناریوی I2C MUX، شامل ارسال شماره کانال از طریق USART، دریافت سه بایت پاسخ، بازسازی مقدار سنسور و بررسی checksum.
- Sensor.h: تعریف رفتار سنسور در سناریوی C2I/I2C، شامل تولید مقدار سنسوری، تبدیل آن به دو بایت داده و یک بایت checksum و قرار دادن داده ها در صف ارسال.
- RangeSensor.h: تعریف رفتار سنسورهای بخش سوم با بازه مقدار قابل تنظیم. در بخش I2C MUX، سنسور صفر در بازه 0..20 و سنسور یک در بازه 50..100 مقدار تولید می کند تا خروجی دو کانال از هم قابل تشخیص باشد.
- HardDisk.h: تعریف رفتار حافظه فرضی در بخش USART، شامل دریافت آدرس از میکروکنترلر، جستجوی مقدار متناظر در حافظه داخلی و ارسال پاسخ از طریق USART.
- I2CMultiplexer.h: تعریف رفتار اجرایی مالتی پلکسر، شامل دریافت شماره کانال از میکروکنترلر، انتخاب کانال، درخواست سه بایت داده از سنسور همان کانال و ارسال پاسخ به میکروکنترلر.
- part1_main.cpp: اجرای مستقل سناریوی C2I/I2C بین ESP8266 و VL530X.
- part2_main.cpp: اجرای مستقل سناریوی USART بین ESP8266 و Usb/HardDisk.
- part3_main.cpp: اجرای مستقل سناریوی I2C MUX شامل ESP8266، I2CMux و دو سنسور VL530X هم آدرس روی دو کانال جداگانه.
- I2C_out.txt: خروجی تست مستقل ارتباط C2I/I2C.
- USART_out.txt: خروجی تست مستقل ارتباط USART.
- I2CMUX_out.txt: خروجی تست مستقل بخش I2C MUX برای بررسی انتخاب کانال های صفر و یک و دریافت داده از دو سنسور هم آدرس.

۲۸ تقسیم کار اعضای گروه

تقسیم کار پروژه مطابق سه بخش اصلی تمرین، یعنی ارتباط C2I/I2C، ارتباط USART و بخش I2C MUX، به صورت زیر انجام شد:

جدول ۴: تقسیم کار اعضای گروه بر اساس سه بخش پروژه		
عضو گروه	بخش اصلی	توضیح کار انجام شده
فرشته باقری	بخش اول: C2I/I2C	تکمیل منطق پروتکل آدرس محور، طراحی رفتار master/slave، مدیریت آدرس 0x29، ارسال و دریافت ACK، انتقال سه بایت داده سنسور و بررسی خروجی اختصاصی I2C.
گلبو رشیدی	تکمیل Sketch ها و بخش سوم: I2C MUX	تکمیل رفتار میکروکنترلر و سنسور، تولید داده سنسور، محاسبه و بررسی checksum، پیاده سازی سناریوی I2C MUX، طراحی منطق انتخاب کانال، دریافت داده از دو سنسور هم آدرس و تحلیل خروجی I2CMUX_out.txt.
مهراد لیویان	بخش دوم: USART Full Duplex	پیاده سازی پروتکل USART، طراحی فریم شامل start bit، هشت بیت داده و stop bit، مدیریت ماشین حالت های RX/TX، تعریف بورد Usb و تست مسیر رفت و برگشت داده.
محمد امانلو	اتصال اجزا، HardDisk و تست نهایی	تکمیل بخش ارتباطی HardDisk، اتصال پایه ها و لینک ها در main، اجرای سناریوهای تست مستقل و ترکیبی، بررسی خروجی های I2C، USART و I2C MUX و آماده سازی گزارش نهایی.

۲۹ جمع بندی

در این تمرین، ما از یک زیرساخت آماده برای شبیه سازی سامانه های نهفته استفاده کردیم و منطق ارتباطی مورد نیاز را روی آن تکمیل کردیم. بخش قابل توجهی از کدهای پایه، مانند کلاس های مربوط به بورد، پردازنده، پایه، لینک، صف، واحدهای داده و Sketch ها از قبل آماده بودند. کار اصلی ما تغییر بنیادی این زیرساخت نبود، بلکه استفاده از آن برای پیاده سازی و تکمیل پروتکل های ارتباطی و رفتار اجزای مورد نیاز تمرین بود. در بخش اول، ارتباط I2C/C2I بین ESP8266 و VL530X تکمیل شد. میکروکنترلر به عنوان master آدرس سنسور را ارسال می کند، سنسور به عنوان slave آدرس را بررسی می کند، در صورت تطابق ACK می دهد و سپس داده سنسور را در قالب دو بایت داده و یک بایت checksum در اختیار میکروکنترلر قرار می دهد. در سمت میکروکنترلر، داده بازسازی و اعتبارسنجی می شود و فقط در صورت معتبر بودن گزارش می شود. در بخش دوم، پروتکل USART برای ارتباط دوطرفه بین میکروکنترلر و هارددیسک فرضی تکمیل شد. در این بخش، هر بایت در قالب فریم شامل start bit، هشت بیت داده و stop bit ارسال می شود. میکروکنترلر یک آدرس برای HardDisk می فرستد و مقدار متناظر را از حافظه داخلی خوانده و از طریق USART برمی گرداند.

در بخش سوم، یک I2C MUX برای حل مشکل سنسورهای هم آدرس طراحی و تست شد. در این سناریو، دو سنسور VL530X با آدرس یکسان 0x29 روی دو کانال جداگانه قرار گرفتند. میکروکنترلر از طریق USART شماره کانال را برای MUX ارسال می کند. سپس MUX همان کانال را انتخاب می کند، از طریق I2C داده

سنسور همان کانال را می خواند و سه بایت داده را از طریق USART به میکروکنترلر برمی گرداند. این بخش نشان داد که با جدا کردن مسیرهای فیزیکی، می توان چند قطعه با آدرس یکسان را بدون تداخل در یک سیستم استفاده کرد.

همچنین در تابع main، اتصال های فیزیکی موردنیاز بین بوردها برقرار شد و Sketch های هر قطعه اجرا شدند. به این ترتیب، شبیه ساز توانست هم ارتباط مستقیم I2C/C2I بین میکروکنترلر و سنسور، هم ارتباط سریال USART بین میکروکنترلر و حافظه فرضی، و هم ارتباط چندکاناله از طریق I2C MUX را مدل کند. خروجی اجرای برنامه نیز نشان داد که هر سه بخش اصلی درست کار می کنند. پیام های I2C نشان دهنده ارسال آدرس، دریافت ACK و انتقال داده سنسور هستند. پیام های USART نشان می دهند که آدرس های ارسال شده توسط میکروکنترلر در هارددیسک خوانده می شوند و داده متناظر دوباره به میکروکنترلر برمی گردد. خروجی I2C MUX نیز نشان می دهد که کانال های صفر و یک به درستی انتخاب شده اند و داده سنسورهای هم آدرس بدون تداخل دریافت شده است.

به طور کلی، پیاده سازی انجام شده نشان داد که چارچوب شبیه ساز می تواند علاوه بر مدل کردن اتصال فیزیکی قطعات، رفتار نرم افزاری و قراردادی ارتباط میان آن ها را نیز مدل کند. این موضوع برای فهم سامانه های بیدرنگ نهفته مهم است، چون در چنین سامانه هایی ارتباط صحیح میان قطعات به ترکیب درست سخت افزار، زمان بندی، پروتکل، انتخاب مسیر ارتباطی و اعتبارسنجی داده وابسته است.